```
$ whoami
apprenti-hacker-ethique
$ cat objectifs.txt
comprendre > proteger > tester avec autorisation
$ ./progresser --jours 30
```

$ cat cours_complet.pdf

Devenir hacker éthique : le cours complet

Les fondamentaux, de zéro, pour comprendre, protéger et tester avec autorisation. Compagnon du programme de 30 jours.

Édition française · conçue pour être suivie depuis un téléphone · 2026

Sommaire

Comment utiliser ce cours 9

Les encadrés 9

Ce que ce cours est — et ce qu'il n'est pas 10

Chapitre 1 — Qu'est-ce que le hacking éthique ? 12

Le mot « hacker » 12

Les « chapeaux » 12

Les métiers de la cybersécurité offensive et défensive 13

La triade CIA 14

Les qualités d'un bon hacker éthique 14

Chapitre 2 — Le cadre légal et éthique 16

Le principe fondamental 16

Ce qui rend un test légal 17

Les terrains de jeu légaux 17

La divulgation responsable 18

Le bug bounty 19

Le code éthique du professionnel 19

Chapitre 3 — Ton laboratoire de poche 20

Pourquoi Linux ? 20

Terminal, shell, commande 20

Installer Termux 21

Les paquets 22

Lire la documentation 22

Chapitre 4 — Maîtriser la ligne de commande Linux 24

L'arborescence 24

Chemins absolus et relatifs . . . 25
Les commandes de navigation et de gestion . . . 25
Éditer un fichier avec nano . . . 26
Chercher : grep et find . . . 26
Pipes et redirections : le super-pouvoir de Linux . . . 26
Se connecter à distance : SSH . . . 27

Chapitre 5 — Utilisateurs, groupes et permissions . . . 28

Pourquoi c'est central en sécurité . . . 28
Utilisateurs et groupes . . . 28
Lire les permissions . . . 28
La notation en chiffres . . . 29
Changer les permissions . . . 30
root et sudo . . . 30
Un premier script . . . 30

Chapitre 6 — Les réseaux : modèles et adressage . . . 32

Pourquoi les réseaux d'abord ? . . . 32
Le modèle OSI . . . 32
Le modèle TCP/IP . . . 33
L'encapsulation . . . 33
Les adresses MAC . . . 34
Les adresses IPv4 . . . 34
Adresses privées et NAT . . . 34
Masques et notation CIDR . . . 35
IPv6 . . . 36
Les équipements . . . 36

Chapitre 7 — Ports, protocoles et web . . . 37

TCP et UDP . . . 37

La poignée de main TCP . . . 37
Les ports . . . 38
Les 15 ports à connaître par cœur . . . 38
ICMP et ping . . . 39
DNS : l'annuaire d'internet . . . 39
HTTP : le langage du web . . . 40
HTTPS et TLS . . . 41
Ce qui se passe quand tu ouvres un site . . . 41

Chapitre 8 — Scanner un réseau avec Nmap . . . 42

À quoi sert un scan ? . . . 42
Nmap . . . 42
Les états d'un port . . . 43
Les options essentielles . . . 43
Ton premier scan, pas à pas . . . 44
Scanner ton propre réseau . . . 44
Interpréter et réfléchir . . . 44

Chapitre 9 — Sécurité Wi-Fi et réseaux locaux . . . 46

Comment fonctionne le Wi-Fi . . . 46
L'évolution des protocoles de sécurité . . . 46
Les risques principaux . . . 47
Sécuriser un réseau domestique ou un hotspot . . . 47
Se protéger sur un réseau public . . . 48

Chapitre 10 — La méthodologie d'un test d'intrusion . . . 50

Pourquoi une méthode ? . . . 50
Les cinq phases . . . 50
Les types de tests . . . 51
Audit, pentest, Red Team, bug bounty . . . 51

La kill chain . . . 52
Prendre des notes : une compétence à part entière . . . 52

Chapitre 11 — La reconnaissance et l'OSINT . . . 53

Reconnaissance passive et active . . . 53
Ce que l'on cherche sur une organisation . . . 53
Les sources classiques . . . 54
Les opérateurs de recherche . . . 54
Faire son propre audit d'exposition . . . 55
L'énumération : la reconnaissance active . . . 56

Chapitre 12 — Comment fonctionne une application web . . . 57

Le modèle client-serveur . . . 57
Le front-end et le back-end . . . 57
Les paramètres . . . 58
Les cookies et les sessions . . . 58
Les API . . . 59
Le proxy d'interception . . . 59
Les en-têtes de sécurité . . . 60

Chapitre 13 — L'OWASP Top 10 . . . 61

Qu'est-ce que l'OWASP ? . . . 61
Les grandes catégories . . . 61
Quelques exemples pour fixer les idées . . . 62
Les protections qui reviennent toujours . . . 63

Chapitre 14 — Les injections : SQL et XSS . . . 64

Le principe de toute injection . . . 64
Le langage SQL en deux minutes . . . 64
Comment naît une injection SQL . . . 64
La vraie solution : les requêtes préparées . . . 65

L'injection de commandes . . . 66
Le XSS (Cross-Site Scripting) . . . 66
Se protéger du XSS . . . 67

Chapitre 15 — Authentification, sessions et contrôle d'accès . 68

Authentification et autorisation : deux choses différentes . . . 68
Les failles d'authentification courantes . . . 68
La gestion des sessions . . . 69
L'IDOR : la faille la plus simple et la plus fréquente . . . 69
L'élévation de privilèges horizontale et verticale . . . 70
L'authentification multifacteur (MFA) . . . 70

Chapitre 16 — Mots de passe et hachage . . . 71

Comment un site devrait stocker ton mot de passe . . . 71
Les propriétés d'une fonction de hachage . . . 71
Hachage, chiffrement, encodage : ne plus jamais confondre . . . 72
Alors, comment « casse-t-on » un mot de passe ? . . . 72
Le sel et les algorithmes lents . . . 73
Ce qui fait un bon mot de passe . . . 73

Chapitre 17 — Les bases de la cryptographie . . . 75

Le vocabulaire . . . 75
Le chiffrement symétrique . . . 75
Le chiffrement asymétrique . . . 75
La signature numérique . . . 76
Le meilleur des deux mondes . . . 76
Les certificats et les autorités . . . 76
Les clés SSH en pratique . . . 77
Base64 : l'exemple type de fausse protection . . . 77

Chapitre 18 — L'élévation de privilèges sous Linux . . . 78

De quoi parle-t-on ? . . . 78
Les grandes familles de causes . . . 78
Le bit SUID . . . 79
Comment un auditeur vérifie un système . . . 79
Se défendre . . . 80

Chapitre 19 — Penser en défenseur : la Blue Team . . . 81

Pourquoi un attaquant éthique doit penser en défenseur . . . 81
La défense en profondeur . . . 81
Les journaux (logs) . . . 82
Le SOC et le SIEM . . . 82
La réponse à incident . . . 82
Les règles d'hygiène de base . . . 83

Chapitre 20 — L'ingénierie sociale . . . 84

Pirater l'humain . . . 84
Les leviers psychologiques . . . 84
Les formes courantes . . . 84
Reconnaître un message piège . . . 85
L'ingénierie sociale dans les tests autorisés . . . 85

Chapitre 21 — CTF et rédaction de rapport . . . 87

Les CTF . . . 87
La méthode pour un défi . . . 88
Pourquoi le rapport est si important . . . 88
La structure d'un rapport . . . 88
Évaluer la gravité . . . 89
Rédiger une vulnérabilité : un modèle . . . 89
Les règles d'écriture . . . 90

Chapitre 22 — Après les 30 jours : construire ta carrière . . . 91

Où tu en es . . . 91
Un parcours possible sur six mois . . . 91
L'obstacle de l'ordinateur . . . 92
Les certifications . . . 92
Se construire un portfolio . . . 93
Les communautés . . . 93
Le mot de la fin . . . 94

Annexe A — Aide-mémoire des commandes . . . 95
Navigation et fichiers . . . 95
Recherche et traitement de texte . . . 95
Utilisateurs et permissions . . . 96
Réseau . . . 96
Hachage et encodage . . . 97
Termux . . . 97

Annexe B — Glossaire . . . 98

Annexe C — Corrigés des questions de validation . . . 101
Phase 1 . . . 101
Phase 2 . . . 101
Phase 3 . . . 102
Phase 4 . . . 102

INTRODUCTION

Comment utiliser ce cours

Ce cours accompagne le programme de 30 jours. Chaque chapitre correspond à une ou plusieurs journées du programme : quand le programme te dit « Cours, chapitre 6 », c'est ici que tu viens lire. Tu peux aussi le lire d'une traite, puis l'utiliser comme ouvrage de référence.

Il est organisé en quatre parties qui suivent les quatre phases du programme :

Partie	Chapitres	Jours du programme
I — Fondations	1 à 5	1 à 7
II — Réseaux	6 à 9	8 à 14
III — Méthode et Web	10 à 15	15 à 22
IV — Systèmes et défense	16 à 22	23 à 30
Annexes	Aide-mémoire, glossaire	À garder sous la main

Les encadrés

Tout au long du cours, tu trouveras des encadrés de couleur. Ils ont chacun un rôle précis :

> **À RETENIR**
>
> Les encadrés **À RETENIR** résument l'idée qu'il faut absolument avoir en tête à la fin du chapitre.

ATTENTION

Les encadrés **ATTENTION** signalent un piège technique ou, plus souvent, une limite légale ou éthique.

ASTUCE TÉLÉPHONE

Les encadrés **ASTUCE TÉLÉPHONE** te montrent comment faire depuis ton smartphone, avec Termux ou ton navigateur.

EXERCICE

Les encadrés **EXERCICE** te proposent quelque chose à faire tout de suite. Ne les saute pas : c'est en pratiquant qu'on retient.

Ce que ce cours est — et ce qu'il n'est pas

Ce cours t'apprend à **comprendre** comment les systèmes sont attaqués, pour pouvoir les **protéger** et les **tester avec autorisation**. Il te donne les fondamentaux qu'on attend d'un débutant sérieux : Linux, réseaux, méthodologie, sécurité web, mots de passe, cryptographie, défense, rédaction de rapports.

Il ne contient volontairement **pas** de recettes d'attaque prêtes à l'emploi, ni de code malveillant. Ce n'est pas un oubli : un bon professionnel n'apprend pas des recettes, il apprend des **mécanismes**. Les recettes changent tous les mois ; les mécanismes restent valables pendant des décennies. Pour la pratique offensive concrète, tu utiliseras des laboratoires légaux conçus pour ça, qui te guident pas à pas dans un cadre sûr.

À RETENIR

Le meilleur hacker éthique n'est pas celui qui connaît le plus d'outils. C'est celui qui comprend le mieux **comment les choses fonctionnent** — parce que toute faille est simplement un endroit où un système ne fonctionne pas comme ses concepteurs le pensaient.

CHAPITRE 1 · JOUR 1

Qu'est-ce que le hacking éthique ?

Le mot « hacker »

À l'origine, dans les années 1960, un *hacker* était simplement quelqu'un de passionné qui aimait comprendre en profondeur le fonctionnement des machines et les pousser au-delà de ce qui était prévu. Le mot n'avait rien de criminel. C'est avec la médiatisation des premières intrusions informatiques, dans les années 1980, que le grand public l'a associé au piratage.

Aujourd'hui, dans le milieu professionnel, on distingue les hackers selon **leur intention et leur autorisation**, pas selon leurs compétences techniques. Un pirate et un pentesteur peuvent utiliser exactement les mêmes outils et les mêmes techniques. Ce qui les sépare, c'est un document signé.

Les « chapeaux »

Profil	Autorisation	Intention	Statut
White hat (chapeau blanc)	Oui, écrite	Protéger, améliorer	Légal — c'est le métier visé
Black hat (chapeau noir)	Non	Voler, nuire, extorquer	Illégal
Grey hat (chapeau gris)	Non	Souvent « bonne », mais sans accord	Illégal malgré l'intention

Le cas du *grey hat* est important, car c'est le piège dans lequel tombent beaucoup de débutants enthousiastes. Trouver une faille

sur le site d'une entreprise sans autorisation, puis la lui signaler en pensant rendre service, reste une intrusion. Certaines entreprises remercient ; d'autres portent plainte. Tu n'as aucun moyen de savoir à l'avance laquelle tu auras en face de toi. La seule position sûre est la position blanche.

Les métiers de la cybersécurité offensive et défensive

La cybersécurité est un domaine immense. Le hacking éthique n'en est qu'une partie, mais ses compétences sont utiles partout.

- **Pentesteur (testeur d'intrusion)** : il est payé pour attaquer un système défini, dans un temps limité, et rédiger un rapport des failles trouvées avec leurs corrections.
- **Red Team** : une équipe qui simule une attaque réaliste et complète contre une organisation, souvent sur plusieurs semaines, pour tester sa capacité à détecter et réagir.
- **Blue Team** : les défenseurs. Ils surveillent, détectent, répondent aux incidents. Analyste SOC (centre opérationnel de sécurité) est souvent le premier poste dans ce domaine.
- **Purple Team** : une collaboration entre Red et Blue, où l'attaque sert directement à améliorer la détection.
- **Chasseur de bugs (bug hunter)** : un indépendant qui cherche des failles sur des programmes de *bug bounty* publics, et est rémunéré par faille validée.
- **Analyste en sécurité applicative** : il audite le code et l'architecture des applications avant leur mise en production.

> **À RETENIR**
>
> Tu n'as pas besoin de choisir maintenant. Les fondamentaux de ce cours sont communs à tous ces métiers. Beaucoup de professionnels commencent côté défense puis passent à l'offensive, ou l'inverse.

La triade CIA

Toute la sécurité de l'information repose sur trois propriétés qu'on cherche à protéger. On les appelle la **triade CIA** (de l'anglais *Confidentiality, Integrity, Availability*) ou **DIC** en français.

Propriété	Question	Exemple d'atteinte
Confidentialité	Seules les bonnes personnes peuvent-elles voir l'information ?	Fuite de la base clients d'une boutique en ligne
Intégrité	L'information est-elle exacte et non modifiée ?	Quelqu'un change le montant d'un virement
Disponibilité	L'information est-elle accessible quand on en a besoin ?	Un site rendu inaccessible par une attaque par saturation

Chaque fois que tu étudieras une faille dans ce cours, demande-toi : **laquelle des trois propriétés est menacée ?** Ce réflexe te servira aussi bien pour évaluer la gravité d'une faille que pour rédiger un rapport.

On ajoute souvent deux notions complémentaires : l'**authentification** (prouver qui l'on est) et la **traçabilité** (savoir qui a fait quoi, et quand).

Les qualités d'un bon hacker éthique

Contrairement à une idée reçue, il n'y a pas besoin d'être un génie des mathématiques. Les qualités qui comptent vraiment sont :

1. **La curiosité** : se demander sans cesse « comment ça marche ? » et « que se passe-t-il si… ? ».
2. **La patience** : la majorité du temps se passe à chercher, lire, essayer, échouer.
3. **La rigueur** : noter tout ce qu'on fait, être méthodique, ne pas sauter d'étapes.

4. **L'intégrité** : on te confie l'accès à des systèmes et des données sensibles. La confiance est ta principale valeur professionnelle.
5. **La communication** : trouver une faille ne sert à rien si tu ne sais pas l'expliquer clairement à ceux qui doivent la corriger.

EXERCICE

Dans ton journal de bord, note pour chacune des cinq qualités un exemple de ta vie où tu l'as déjà montrée. Tu as plus de bases que tu ne le crois.

CHAPITRE 2 · JOUR 2

Le cadre légal et éthique

ATTENTION

Ce chapitre n'est pas un conseil juridique. Les lois varient d'un pays à l'autre et évoluent. Renseigne-toi toujours sur la législation du pays où tu te trouves et de celui où se trouve le système concerné. En cas de doute réel, demande à un juriste.

Le principe fondamental

Dans presque tous les pays du monde, la loi punit le fait d'**accéder sans autorisation** à un système informatique, de s'y **maintenir**, d'en **modifier** les données ou d'en **perturber** le fonctionnement. Beaucoup de législations punissent aussi le simple fait de **détenir ou diffuser des outils** conçus principalement pour commettre ces infractions, sans motif légitime.

En France, ces infractions sont prévues par les articles 323-1 et suivants du Code pénal (héritiers de la loi Godfrain de 1988). De nombreux pays d'Afrique francophone se sont dotés de lois sur la cybercriminalité qui reprennent des principes très proches, souvent inspirés de la Convention de Budapest ou de la Convention de l'Union africaine sur la cybersécurité (dite de Malabo).

Retiens que dans la plupart des cas :

- l'intention n'efface pas l'infraction (« je voulais juste tester » n'est pas une défense) ;
- l'absence de dégâts n'efface pas l'infraction (entrer suffit) ;
- un système « mal protégé » reste protégé par la loi.

Ce qui rend un test légal

Un test d'intrusion légal repose sur un **accord écrit** du propriétaire du système. Cet accord définit précisément :

Élément	Ce qu'il précise
Le périmètre (scope)	Quelles adresses IP, quels domaines, quelles applications sont concernés — et lesquels sont exclus
Les dates et horaires	Quand les tests peuvent avoir lieu
Les techniques autorisées	Par exemple : ingénierie sociale oui ou non, tests de saturation oui ou non
Les contacts	Qui prévenir en cas de problème ou de découverte critique
La confidentialité	Ce que tu peux faire des données vues pendant le test

Ce document est souvent appelé **lettre d'autorisation** ou, dans le jargon, la « *carte de sortie de prison* ». Les professionnels l'ont toujours sur eux pendant une mission.

> **ATTENTION**
>
> Une autorisation donnée par quelqu'un qui **n'est pas propriétaire** du système ne vaut rien. Un employé ne peut pas t'autoriser à tester les serveurs de son entreprise s'il n'en a pas le pouvoir. Un ami ne peut pas t'autoriser à tester le Wi-Fi de son immeuble s'il ne le gère pas.

Les terrains de jeu légaux

Heureusement, il existe aujourd'hui énormément d'endroits où tu peux t'entraîner en toute légalité :

- **Tes propres appareils** : ton téléphone, ta box, ton routeur, tes comptes.

- **Les plateformes d'entraînement** : TryHackMe, Hack The Box, Root Me, PortSwigger Web Security Academy, OverTheWire, picoCTF. Elles mettent à ta disposition des machines volontairement vulnérables.
- **Les applications vulnérables à installer soi-même** : DVWA, OWASP Juice Shop, etc. (plus faciles avec un ordinateur, mais certaines plateformes les hébergent pour toi).
- **Les CTF (Capture The Flag)** : des compétitions où tout est autorisé dans le cadre défini par l'organisateur.
- **Les programmes de bug bounty** : des entreprises qui autorisent publiquement des tests sur un périmètre défini, selon des règles précises.

La divulgation responsable

Que faire si tu découvres une faille par hasard, sans l'avoir cherchée — par exemple, une page qui affiche des données d'autres utilisateurs en changeant un chiffre dans l'adresse ?

1. **Arrête-toi immédiatement.** N'explore pas davantage, ne télécharge rien, ne « vérifies » pas en allant plus loin.
2. **Note** ce que tu as vu et comment, sans conserver de données personnelles.
3. **Cherche le canal officiel** de signalement : un fichier `/.well-known/security.txt` sur le site, une page « Sécurité » ou « Divulgation responsable », un programme de bug bounty.
4. **Signale** de façon factuelle et courtoise, sans aucune demande d'argent (une demande de paiement contre le silence s'appelle de l'extorsion).
5. **Laisse un délai raisonnable** de correction avant toute communication publique, en accord avec l'organisation.

Le bug bounty

Les programmes de *bug bounty* sont des invitations publiques à chercher des failles, contre récompense. Des plateformes comme HackerOne, Bugcrowd, Intigriti ou YesWeHack les centralisent. Chaque programme a sa propre politique, qu'il faut lire **entièrement** avant de commencer : périmètre, types de failles acceptées, techniques interdites.

À RETENIR

Le bug bounty n'est pas le bon point de départ pour un débutant. Les cibles sont testées par des milliers de chercheurs expérimentés. Commence par les laboratoires ; le bug bounty viendra naturellement quand tu maîtriseras les bases.

Le code éthique du professionnel

Les organismes de certification demandent à leurs membres de respecter un code de conduite. Les principes reviennent toujours :

- rester dans le périmètre autorisé ;
- protéger la confidentialité des données rencontrées ;
- ne jamais utiliser ses accès à des fins personnelles ;
- signaler honnêtement tout ce qu'on a trouvé, y compris ses propres erreurs pendant le test ;
- ne pas causer de dommages évitables.

EXERCICE

Rédige dans ton journal ta propre charte en 5 lignes, à la première personne : « Je ne teste que… », « Si je découvre une faille par hasard, je… ». Relis-la le jour 30.

CHAPITRE 3 · JOUR 3

Ton laboratoire de poche

Pourquoi Linux ?

Linux est un système d'exploitation libre, utilisé par l'immense majorité des serveurs web, des équipements réseau, des objets connectés… et par Android lui-même, dont le cœur (le *noyau*) est un noyau Linux. En sécurité, il est incontournable pour trois raisons :

- **Les cibles** : la plupart des serveurs que tu testeras tourneront sous Linux.
- **Les outils** : la grande majorité des outils de sécurité sont conçus pour Linux.
- **La transparence** : tout y est fichier, tout y est configurable, tout s'apprend.

Les distributions spécialisées comme **Kali Linux** ou **Parrot OS** regroupent des centaines d'outils de sécurité. Elles s'installent sur un ordinateur. Sur téléphone, tu vas utiliser **Termux**, qui te donne un environnement Linux sans avoir besoin de « rooter » ton appareil.

Terminal, shell, commande

Ces trois mots sont souvent confondus :

Terme	Ce que c'est	Analogie
Terminal	La fenêtre où tu tapes du texte	Le téléphone
Shell	Le programme qui comprend tes commandes (souvent `bash`)	L'interprète au bout du fil
Commande	L'instruction que tu donnes	La phrase que tu dis

Quand tu ouvres Termux, tu vois une **invite de commande** (prompt) qui se termine par `$`. Le `$` signifie que tu es un utilisateur normal. Sur un système Linux classique, un `#` signifierait que tu es `root`, l'administrateur tout-puissant.

Installer Termux

1. Télécharge **F-Droid** depuis son site officiel (f-droid.org) et installe-le. Android te demandera d'autoriser l'installation depuis cette source.
2. Dans F-Droid, cherche **Termux** et installe-le.
3. Ouvre Termux et attends la fin de l'installation initiale.
4. Mets à jour les paquets, puis installe les outils de base :

```
pkg update && pkg upgrade
pkg install nano curl openssh git python man
termux-setup-storage
```

La dernière commande donne à Termux l'accès au stockage de ton téléphone (un dossier `storage` apparaîtra dans ton répertoire personnel).

> **ASTUCE TÉLÉPHONE**
>
> Termux affiche une ligne de touches supplémentaires au-dessus du clavier (Ctrl, Alt, Tab, flèches, Échap). Si elle n'apparaît pas, glisse vers la gauche depuis le bord de l'écran, ou augmente le volume + Q. Les combinaisons à connaître : **Ctrl+C** pour arrêter une commande, **Tab** pour compléter automatiquement un nom, **flèche haut** pour rappeler la commande précédente.

Les paquets

Sous Linux, on n'installe pas les logiciels en téléchargeant des fichiers sur des sites web au hasard. On utilise un **gestionnaire de paquets**, qui va chercher les logiciels dans des dépôts officiels et vérifiés. Dans Termux, c'est `pkg` (une surcouche de `apt`, le gestionnaire de Debian et Ubuntu).

Commande	Effet
`pkg search nom`	Chercher un paquet
`pkg install nom`	Installer
`pkg uninstall nom`	Désinstaller
`pkg update`	Mettre à jour la liste des paquets disponibles
`pkg upgrade`	Mettre à jour les paquets installés
`pkg list-installed`	Lister ce qui est installé

> **ATTENTION**
>
> N'installe jamais des « outils de hacking tout-en-un » trouvés sur YouTube ou Telegram avec des scripts du genre « copie-colle cette commande ». Beaucoup contiennent eux-mêmes des logiciels malveillants qui visent… les débutants. Ne lance que ce que tu comprends, depuis des sources officielles.

Lire la documentation

Le réflexe le plus important de tout professionnel : **lire la doc**. Presque toutes les commandes acceptent l'option `--help`, et beaucoup ont une page de manuel :

```
ls --help
man ls
```

Dans `man`, utilise les flèches pour défiler, `/mot` pour chercher, et `q` pour quitter.

EXERCICE

Lance `whoami`, `pwd`, `uname -a`, `date` et `echo "Bonjour JRK"`. Puis lis `man echo` et trouve à quoi sert l'option `-n`.

CHAPITRE 4 · JOURS 4 ET 5

Maîtriser la ligne de commande Linux

L'arborescence

Sous Linux, tous les fichiers sont rangés dans une seule grande arborescence qui part de la **racine** `/`. Il n'y a pas de « C: » ou « D: » comme sous Windows.

Dossier	Contenu
`/`	La racine, point de départ de tout
`/home`	Les dossiers personnels des utilisateurs
`/root`	Le dossier personnel de l'administrateur
`/etc`	Les fichiers de configuration du système
`/var/log`	Les journaux (logs)
`/tmp`	Les fichiers temporaires, effacés régulièrement
`/bin`, `/usr/bin`	Les programmes (commandes)

ASTUCE TÉLÉPHONE

Dans Termux, ton dossier personnel n'est pas `/home/toi` mais un chemin plus long propre à Android. Peu importe : le symbole `~` désigne toujours ton dossier personnel, où que tu sois. `cd ~` t'y ramène.

Chemins absolus et relatifs

- Un **chemin absolu** part de la racine : `/etc/passwd`. Il marche de n'importe où.
- Un **chemin relatif** part de l'endroit où tu es : `notes/jour4.txt`.
- `.` désigne le dossier actuel, `..` le dossier parent.

Les commandes de navigation et de gestion

Commande	Rôle	Exemple
`pwd`	Où suis-je ?	`pwd`
`ls`	Lister	`ls -la` (tout, y compris les fichiers cachés, en détail)
`cd`	Changer de dossier	`cd labo`, `cd ..`, `cd ~`
`mkdir`	Créer un dossier	`mkdir -p labo/notes`
`touch`	Créer un fichier vide	`touch test.txt`
`cp`	Copier	`cp a.txt b.txt`
`mv`	Déplacer ou renommer	`mv b.txt notes/`
`rm`	Supprimer	`rm test.txt`, `rm -r dossier`
`cat`	Afficher un fichier	`cat notes.txt`
`less`	Lire page par page	`less long.txt`
`head` / `tail`	Début / fin d'un fichier	`tail -n 20 journal.log`

ATTENTION

`rm` ne met rien à la corbeille. Ce qui est supprimé est perdu. Et `rm -rf` sur le mauvais dossier peut effacer énormément de choses en une fraction de seconde. Relis toujours une commande `rm` avant d'appuyer sur Entrée.

Les fichiers dont le nom commence par un point (`.bashrc`, `.ssh`) sont **cachés** : `ls` seul ne les montre pas, `ls -a` oui. En sécurité, on regarde **toujours** les fichiers cachés : c'est souvent là que se trouvent configurations, historiques et clés.

Éditer un fichier avec nano

`nano` est l'éditeur de texte le plus simple du terminal. `nano fichier.txt` ouvre (ou crée) le fichier. Les raccourcis sont affichés en bas : le `^` signifie Ctrl. **Ctrl+O** puis Entrée pour enregistrer, **Ctrl+X** pour quitter.

Chercher : grep et find

`grep` cherche du **texte à l'intérieur** des fichiers :

```
grep "password" config.txt        # lignes contenant le mot
grep -i "password" config.txt     # sans tenir compte des
majuscules
grep -r "password" ~/labo         # dans tout un dossier
grep -v "commentaire" fichier     # lignes NE contenant PAS
le mot
```

`find` cherche des **fichiers** selon leur nom, leur taille, leur type :

```
find ~/labo -name "*.txt"         # tous les fichiers .txt
find ~ -type d -name "notes"      # les dossiers nommés notes
find . -size +1M                  # les fichiers de plus d'1
Mo
```

Pipes et redirections : le super-pouvoir de Linux

La philosophie de Linux est simple : chaque commande fait **une seule chose, bien**, et on les combine.

- Le **pipe** `|` envoie le résultat d'une commande vers une autre.
- `>` écrit le résultat dans un fichier (en **écrasant** son contenu).

- `>>` ajoute le résultat à la fin d'un fichier.

```
ls -la | grep ".txt"              # ne garder que les .txt
cat acces.log | grep "404" | wc -l   # compter les erreurs 404
echo "jour 5 terminé" >> journal.txt
```

D'autres commandes utiles à combiner : `wc -l` (compter les lignes), `sort` (trier), `uniq` (dédoublonner), `cut` (découper des colonnes), `strings` (extraire le texte lisible d'un fichier binaire), `file` (deviner le type d'un fichier).

À RETENIR

Les niveaux Bandit d'OverTheWire sont construits exactement autour de ces commandes. Chaque niveau t'oblige à en découvrir une nouvelle. C'est le meilleur entraînement qui existe pour devenir à l'aise dans un terminal.

Se connecter à distance : SSH

SSH (*Secure Shell*) permet d'ouvrir un terminal sur une autre machine, à travers le réseau, de façon chiffrée. C'est la façon standard d'administrer un serveur Linux. Pour Bandit :

```
ssh bandit0@bandit.labs.overthewire.org -p 2220
```

`bandit0` est le nom d'utilisateur, l'adresse est celle du serveur, et `-p 2220` indique le port (on verra ce qu'est un port au chapitre 7). Le mot de passe du niveau 0 est indiqué sur le site. Tape `exit` pour te déconnecter.

EXERCICE

Réussis les niveaux 0 à 5 de Bandit. Pour chaque niveau, note dans ton journal **la commande clé** et **pourquoi** elle a fonctionné.

CHAPITRE 5 · JOUR 6

Utilisateurs, groupes et permissions

Pourquoi c'est central en sécurité

Linux est un système **multi-utilisateurs**. Chaque fichier appartient à quelqu'un, et chaque action est faite au nom de quelqu'un. Le modèle de permissions décide qui a le droit de faire quoi. Une très grande partie des failles d'élévation de privilèges (chapitre 18) vient simplement de **permissions mal réglées**.

Utilisateurs et groupes

- Chaque utilisateur a un nom et un numéro (UID). `root` a l'UID 0 : il a tous les droits.
- Chaque utilisateur appartient à un ou plusieurs **groupes**, qui permettent de partager des droits.
- `id` affiche ton identité, `groups` tes groupes.
- Le fichier `/etc/passwd` liste les utilisateurs du système. Malgré son nom, il ne contient plus les mots de passe depuis longtemps : leurs empreintes sont dans `/etc/shadow`, lisible uniquement par root.

Lire les permissions

`ls -l` affiche une ligne comme celle-ci :

```
-rwxr-x---  1 alice  devs  2048  oct 30  script.sh
```

Les dix premiers caractères se lisent ainsi :

Position	Signification
1	Type : `-` fichier, `d` dossier, `l` lien
2 à 4	Droits du **propriétaire** (ici `rwx`)
5 à 7	Droits du **groupe** (ici `r-x`)
8 à 10	Droits des **autres** (ici `---`)

Et les lettres :

Lettre	Sur un fichier	Sur un dossier
`r` (read)	Lire le contenu	Lister les fichiers
`w` (write)	Modifier	Créer / supprimer des fichiers dedans
`x` (execute)	Exécuter comme programme	Entrer dans le dossier

Ici : Alice peut tout faire, les membres du groupe `devs` peuvent lire et exécuter, les autres rien.

La notation en chiffres

Chaque droit vaut un nombre : **r = 4, w = 2, x = 1**. On additionne pour chaque catégorie.

Chiffres	Lettres	Usage typique
755	`rwxr-xr-x`	Programme que tout le monde peut lancer
644	`rw-r--r--`	Fichier lisible par tous, modifiable par le propriétaire
700	`rwx------`	Dossier strictement privé
600	`rw-------`	Fichier secret (clé SSH privée par exemple)
777	`rwxrwxrwx`	Tout le monde fait tout — **presque toujours une erreur**

Changer les permissions

```
chmod 750 script.sh       # en chiffres
chmod +x script.sh        # ajouter l'exécution
chmod o-r secret.txt      # retirer la lecture aux "autres"
chown alice:devs fichier  # changer propriétaire et groupe (
souvent réservé à root)
```

root et sudo

Travailler en permanence en tant que root est dangereux : la moindre erreur a des conséquences sur tout le système, et tout programme lancé a tous les droits. On travaille donc en utilisateur normal, et on utilise `sudo` pour lancer **ponctuellement** une commande avec des droits élevés. Le fichier `/etc/sudoers` définit qui a le droit d'utiliser `sudo`, et pour quelles commandes.

Un premier script

Un script est un fichier texte contenant une suite de commandes.

```
nano bonjour.sh
```

Écris dedans :

```
#!/bin/bash
echo "Bonjour, je suis $(whoami)"
echo "Nous sommes le $(date)"
echo "Je suis dans $(pwd)"
```

La première ligne, appelée *shebang*, indique quel programme doit lire le script. Enregistre, puis :

```
chmod +x bonjour.sh
./bonjour.sh
```

Le `./` signifie « le fichier qui est ici, dans ce dossier ».

À RETENIR

Les permissions répondent à trois questions : **qui** (propriétaire, groupe, autres), **quoi** (lire, écrire, exécuter), **sur quoi** (fichier ou dossier). Un fichier sensible lisible par « les autres », un script modifiable par tout le monde mais exécuté par root : ce sont les deux erreurs que tu chercheras toute ta carrière.

AUTO-ÉVALUATION

Que signifient `chmod 644`, `chmod 700` et `chmod 755` ? Pourquoi une clé SSH privée doit-elle être en 600 ? Que risque-t-on si un script lancé automatiquement par root est en 777 ?

CHAPITRE 6 · JOURS 8 ET 9

Les réseaux : modèles et adressage

Pourquoi les réseaux d'abord ?

Presque toute attaque passe par le réseau. Pour comprendre comment on attaque ou défend un système, il faut comprendre comment les données voyagent d'une machine à l'autre. C'est la partie la plus « scolaire » du parcours, mais c'est celle qui fera la plus grande différence entre toi et quelqu'un qui se contente de lancer des outils.

Le modèle OSI

Le modèle OSI découpe la communication réseau en **7 couches**. Chaque couche rend un service à celle du dessus et s'appuie sur celle du dessous. C'est un modèle théorique, mais tout le monde l'utilise pour parler de réseaux.

N°	Couche	Rôle	Exemples
7	Application	Ce que l'utilisateur utilise	HTTP, DNS, SMTP, SSH
6	Présentation	Format, chiffrement, compression	TLS, encodages
5	Session	Ouvrir et maintenir un dialogue	Sessions, RPC
4	Transport	Découper, numéroter, fiabiliser	TCP, UDP (ports)
3	Réseau	Trouver le chemin entre réseaux	IP, ICMP (adresses IP)

N°	Couche	Rôle	Exemples
2	Liaison	Transmettre sur le réseau local	Ethernet, Wi-Fi (adresses MAC)
1	Physique	Signaux électriques, ondes, lumière	Câbles, ondes radio, fibre

ASTUCE TÉLÉPHONE

Moyen mnémotechnique (de la couche 1 à 7) : « **P**our **L**e **R**éseau, **T**out **S**e **P**asse **A**utomatiquement » — Physique, Liaison, Réseau, Transport, Session, Présentation, Application.

Le modèle TCP/IP

Dans la réalité, internet utilise le modèle **TCP/IP**, plus simple, en 4 couches :

Couche TCP/IP	Correspond aux couches OSI	Exemples
Application	5, 6, 7	HTTP, DNS, SSH
Transport	4	TCP, UDP
Internet	3	IP, ICMP
Accès réseau	1, 2	Ethernet, Wi-Fi

L'encapsulation

Quand ton téléphone envoie une requête vers un site web, chaque couche ajoute ses propres informations autour des données, comme on mettrait une lettre dans une enveloppe, puis dans un colis, puis dans un camion :

1. L'application prépare la requête HTTP (la lettre).
2. La couche transport ajoute un en-tête TCP avec les **ports** source et destination (l'enveloppe).

3. La couche internet ajoute un en-tête IP avec les **adresses IP** source et destination (le colis).
4. La couche d'accès ajoute un en-tête avec les **adresses MAC** pour le prochain saut (le camion).

À l'arrivée, chaque couche retire son enveloppe dans l'ordre inverse : c'est la **désencapsulation**.

Les adresses MAC

Chaque carte réseau possède une **adresse MAC**, écrite en 6 paires hexadécimales, par exemple `3C:22:FB:1A:9E:04`. Elle sert à communiquer **à l'intérieur d'un même réseau local**. Les trois premières paires identifient souvent le fabricant. Les téléphones modernes utilisent par défaut des adresses MAC **aléatoires** pour chaque réseau Wi-Fi, afin de limiter le pistage.

Les adresses IPv4

Une adresse **IPv4** est composée de 4 nombres de 0 à 255 séparés par des points : `192.168.1.25`. Elle identifie une machine sur un réseau et permet d'acheminer les paquets **entre** réseaux.

Il existe environ 4,3 milliards d'adresses IPv4 — c'est beaucoup trop peu pour tous les appareils du monde. Deux solutions ont été trouvées : les adresses privées avec le NAT, et l'IPv6.

Adresses privées et NAT

Certaines plages sont réservées aux réseaux **privés** et ne circulent jamais directement sur internet :

Plage privée	Notation	Usage courant
10.0.0.0 à 10.255.255.255	`10.0.0.0/8`	Grandes entreprises
172.16.0.0 à 172.31.255.255	`172.16.0.0/12`	Entreprises, conteneurs

Plage privée	Notation	Usage courant
192.168.0.0 à 192.168.255.255	`192.168.0.0/16`	Box et routeurs domestiques

Ta box ou ton routeur possède **une seule adresse publique** sur internet, et distribue des adresses privées à tous les appareils de la maison. Quand un appareil sort sur internet, le routeur remplace l'adresse privée par son adresse publique : c'est le **NAT** (*Network Address Translation*). C'est pour ça que `ip addr` et `curl ifconfig.me` ne te donnent pas la même adresse.

Deux adresses spéciales à connaître : `127.0.0.1` (*localhost*, la machine elle-même) et `0.0.0.0` (« toutes les interfaces » dans une configuration de serveur).

Masques et notation CIDR

Une adresse IP se compose d'une partie **réseau** et d'une partie **machine**. Le **masque** indique où est la frontière. La notation **CIDR** écrit ce masque sous la forme `/n`, où *n* est le nombre de bits réservés au réseau (sur 32).

CIDR	Masque	Adresses au total	Machines utilisables
`/24`	255.255.255.0	256	254
`/25`	255.255.255.128	128	126
`/16`	255.255.0.0	65 536	65 534
`/30`	255.255.255.252	4	2

On enlève toujours 2 adresses : la première (l'adresse du **réseau** lui-même) et la dernière (l'adresse de **diffusion**, *broadcast*, qui parle à tout le monde).

À RETENIR

Règle de calcul rapide : un réseau `/n` contient 2 puissance (32 − n) adresses. Pour un `/24` : 2 puissance 8 = 256 adresses, dont 254 utilisables. Dans un réseau `192.168.1.0/24`, les machines vont de `192.168.1.1` à `192.168.1.254`.

IPv6

L'**IPv6** utilise des adresses sur 128 bits, écrites en hexadécimal : `2001:0db8:85a3::8a2e:0370:7334`. Le nombre d'adresses disponibles est astronomique. En sécurité, il ne faut pas l'oublier : un serveur bien protégé en IPv4 est parfois totalement ouvert en IPv6, parce que l'administrateur n'a configuré son pare-feu que pour l'un des deux.

Les équipements

Équipement	Couche	Rôle
Switch (commutateur)	2	Relie les appareils d'un même réseau local
Routeur	3	Relie des réseaux différents, choisit le chemin
Point d'accès Wi-Fi	1–2	Relie les appareils sans fil au réseau
Pare-feu (firewall)	3–7	Filtre le trafic selon des règles

Ta « box » internet combine en général tout ça dans un seul boîtier.

EXERCICE

Dans Termux, tape `ip addr` (ou `ifconfig` si la première ne marche pas). Repère ton adresse IPv4 locale et son `/n`. Combien d'appareils ce réseau peut-il contenir ? Puis `curl ifconfig.me` : c'est l'adresse que voient les sites web.

CHAPITRE 7 · JOURS 10 ET 11

Ports, protocoles et web

TCP et UDP

Au niveau transport, deux protocoles dominent :

	TCP	UDP
Connexion	Établie avant l'envoi	Aucune, on envoie directement
Fiabilité	Garantit l'arrivée et l'ordre	Pas de garantie
Vitesse	Plus lent	Plus rapide
Usages	Web, e-mail, SSH, transferts	Streaming, jeux, appels, DNS
Analogie	Lettre recommandée avec accusé	Carte postale

La poignée de main TCP

Avant d'échanger des données, TCP ouvre une connexion en trois temps, le **three-way handshake** :

1. Le client envoie un paquet **SYN** (« je voudrais te parler »).
2. Le serveur répond **SYN-ACK** (« d'accord, je t'écoute »).
3. Le client confirme avec **ACK** (« c'est parti »).

Si le port est fermé, le serveur répond par un **RST** (« non, personne ici »). S'il y a un pare-feu, il arrive qu'il n'y ait **aucune réponse**. Ce sont exactement ces différences de réaction qu'utilisent les scanners de ports pour déduire l'état d'un port (chapitre 8).

Les ports

Une adresse IP identifie une machine ; un **port** identifie un **service** sur cette machine. Si l'adresse IP est l'adresse d'un immeuble, le port est le numéro de l'appartement. Il en existe 65 535 pour TCP, et autant pour UDP. Les ports de 0 à 1023 sont dits « bien connus » et associés par convention à des services standards.

Les 15 ports à connaître par cœur

Port	Service	Remarque sécurité
21	FTP	Transfert de fichiers, souvent en clair
22	SSH	Administration à distance chiffrée
23	Telnet	Ancêtre de SSH, **tout en clair**
25	SMTP	Envoi d'e-mails
53	DNS	Résolution de noms (UDP et TCP)
80	HTTP	Web non chiffré
110	POP3	Réception d'e-mails
139 / 445	SMB	Partage de fichiers Windows, cible historique
143	IMAP	Réception d'e-mails
443	HTTPS	Web chiffré
3306	MySQL	Base de données, ne doit pas être exposée
3389	RDP	Bureau à distance Windows
5432	PostgreSQL	Base de données
8080	HTTP alternatif	Serveurs de test, interfaces d'administration
8291	Winbox	Administration des routeurs MikroTik

ATTENTION

Le numéro de port n'est qu'une **convention**. Un administrateur peut faire tourner SSH sur le port 2220 (comme Bandit) ou un site web sur le port 22. C'est pourquoi on ne se contente jamais du numéro : on identifie le service réellement présent.

ICMP et ping

ICMP est le protocole de messages de contrôle d'IP. Le plus connu est le **ping** : on envoie un « echo request », la machine répond « echo reply ». Cela permet de savoir si une machine est joignable et combien de temps met le trajet. Beaucoup de pare-feu bloquent le ping : une machine qui ne répond pas au ping n'est donc pas forcément éteinte.

DNS : l'annuaire d'internet

Les humains retiennent des noms (`wikipedia.org`), les machines utilisent des adresses IP. Le **DNS** (*Domain Name System*) fait la traduction. Quand tu tapes une adresse dans ton navigateur, ton téléphone demande à un **résolveur DNS** (souvent celui de ton opérateur) : « quelle est l'IP de ce nom ? ». Le résolveur interroge si besoin une chaîne de serveurs jusqu'à obtenir la réponse.

Type d'enregistrement	Contient
A	L'adresse IPv4 d'un nom
AAAA	L'adresse IPv6
CNAME	Un alias vers un autre nom
MX	Les serveurs de messagerie du domaine
TXT	Du texte libre (vérifications, politiques anti-spam)
NS	Les serveurs qui font autorité pour le domaine

```
pkg install dnsutils
nslookup wikipedia.org
dig wikipedia.org MX
```

HTTP : le langage du web

HTTP est un protocole texte, ce qui le rend facile à lire. Une requête ressemble à ceci :

```
GET /index.html HTTP/1.1
Host: www.exemple.com
User-Agent: Mozilla/5.0 (Android)
Cookie: session=a8f3c1...
```

La première ligne contient la **méthode**, la **ressource** demandée et la version. Viennent ensuite les **en-têtes** (*headers*). Le serveur répond avec un **code de statut**, ses propres en-têtes, puis le contenu.

Méthode	Usage
GET	Récupérer une ressource (paramètres visibles dans l'URL)
POST	Envoyer des données (formulaires, paramètres dans le corps)
PUT / PATCH	Modifier une ressource
DELETE	Supprimer une ressource

Code	Famille	Exemples
1xx	Information	Rare en pratique
2xx	Succès	200 OK, 201 Created
3xx	Redirection	301 Moved Permanently, 302 Found
4xx	Erreur du client	401 non authentifié, 403 interdit, 404 introuvable
5xx	Erreur du serveur	500 erreur interne, 503 indisponible

```
curl -I https://wikipedia.org     # seulement les en-têtes de
la réponse
curl -v https://wikipedia.org     # tout le dialogue, en
détail
```

HTTPS et TLS

HTTPS, c'est HTTP transporté à l'intérieur d'un tunnel chiffré **TLS**. TLS apporte trois garanties : la **confidentialité** (personne ne peut lire les échanges), l'**intégrité** (personne ne peut les modifier) et l'**authentification du serveur** grâce à un **certificat** signé par une autorité de confiance.

ATTENTION

Le cadenas signifie que la connexion est chiffrée, **pas** que le site est honnête. Un site d'hameçonnage peut parfaitement avoir un certificat valide et un cadenas.

Ce qui se passe quand tu ouvres un site

1. Le navigateur demande au **DNS** l'adresse IP du nom.
2. Il ouvre une connexion **TCP** vers le port 443 de cette IP (poignée de main).
3. Il négocie un tunnel **TLS** et vérifie le certificat.
4. Il envoie une requête **HTTP** dans le tunnel.
5. Le serveur répond, et le navigateur affiche la page, en répétant tout ça pour les images, scripts et styles.

AUTO-ÉVALUATION

Sans regarder : sur quels ports tournent SSH, HTTP, HTTPS, DNS et MySQL ? Que signifient les codes 403 et 500 ? Quelle est la différence entre un enregistrement A et un enregistrement MX ?

CHAPITRE 8 · JOUR 12

Scanner un réseau avec Nmap

ATTENTION

Scanner les ports d'une machine qui ne t'appartient pas et pour laquelle tu n'as pas d'autorisation est, selon les pays, au minimum mal vu et souvent illégal ; c'est en tout cas considéré comme le premier pas d'une attaque. N'utilise Nmap que sur : ton propre réseau, `scanme.nmap.org` (avec modération), et les machines des plateformes d'entraînement.

À quoi sert un scan ?

Un scan répond à trois questions : **quelles machines** sont actives, **quels ports** sont ouverts sur ces machines, et **quels services** (et quelles versions) écoutent derrière. C'est la base de l'énumération : on ne peut pas sécuriser — ni tester — ce dont on ne connaît pas l'existence.

Côté défense, les administrateurs scannent régulièrement leurs propres réseaux pour repérer les services exposés par erreur. C'est l'un des usages les plus courants de Nmap.

Nmap

Nmap (*Network Mapper*) est l'outil de référence, libre et gratuit, développé depuis 1997. Il s'installe dans Termux :

```
pkg install nmap
nmap --version
```

ASTUCE TÉLÉPHONE

Sans droits root, Nmap dans Termux ne peut pas utiliser certains types de scans qui fabriquent des paquets « à la main ». Il utilise alors le scan par connexion TCP complète (`-sT`), qui fonctionne parfaitement pour apprendre.

Les états d'un port

État	Signification
open	Un service écoute et accepte les connexions
closed	La machine répond, mais aucun service n'écoute sur ce port
filtered	Aucune réponse claire : un pare-feu filtre probablement
unfiltered	Le port est accessible mais Nmap ne peut dire s'il est ouvert

Les options essentielles

Option	Effet
`nmap cible`	Scan des 1 000 ports TCP les plus courants
`-p 22,80,443`	Seulement ces ports
`-p-`	Les 65 535 ports (long !)
`-sT`	Scan par connexion TCP complète
`-sV`	Détecte le service et sa version
`-sn`	Découverte des machines actives, sans scan de ports
`-T2` à `-T4`	Vitesse (plus lent = plus discret et plus poli)
`-oN fichier.txt`	Enregistre le résultat dans un fichier

Ton premier scan, pas à pas

```
nmap -sT -T3 scanme.nmap.org
```

Tu obtiens une liste de ports avec leur état et le nom du service supposé. Ensuite :

```
nmap -sT -sV -p 22,80 scanme.nmap.org -oN scan1.txt
```

L'option `-sV` interroge les services pour identifier leur logiciel et sa version, par exemple « OpenSSH 6.6.1 ». **C'est l'information la plus précieuse d'un scan** : connaître une version permet de vérifier si elle est affectée par des vulnérabilités publiques connues.

Scanner ton propre réseau

Pour voir les appareils connectés à ton réseau Wi-Fi :

```
nmap -sn 192.168.1.0/24
```

(adapte l'adresse à celle trouvée avec `ip addr`). Tu découvriras peut-être des appareils que tu avais oubliés : télévision, imprimante, caméra, ampoule connectée… Chacun est une porte potentielle.

Interpréter et réfléchir

Un scan ne vaut que par l'analyse qu'on en fait. Pour chaque port ouvert, pose-toi ces questions :

1. Ce service **doit-il** être accessible depuis là où je suis ?
2. Sa **version** est-elle à jour ?
3. Utilise-t-il un protocole **chiffré** ?
4. L'**authentification** est-elle robuste (pas d'identifiants par défaut) ?

À RETENIR

Un bon scan est un scan **utile** et **proportionné**. Les professionnels ne lancent pas tout, tout le temps, à pleine vitesse : ils choisissent leurs options en fonction de ce qu'ils cherchent et de ce que le client a autorisé.

EXERCICE

Scanne la box ou le routeur de ta maison (souvent `192.168.1.1` ou `192.168.0.1`) avec `nmap -sT -sV`. Quels services sont ouverts ? Notes-les : ils serviront pour ton rapport du jour 29.

CHAPITRE 9 · JOUR 13

Sécurité Wi-Fi et réseaux locaux

Comment fonctionne le Wi-Fi

Le Wi-Fi transmet les données par **ondes radio**. Contrairement à un câble, ces ondes traversent les murs et peuvent être captées par n'importe qui à portée. Toute la sécurité du Wi-Fi repose donc sur le **chiffrement** et l'**authentification**.

Un réseau Wi-Fi est identifié par son **SSID** (son nom). Le point d'accès émet régulièrement des « balises » pour annoncer sa présence. Cacher le SSID n'apporte quasiment aucune sécurité : le nom réapparaît dès qu'un appareil se connecte.

L'évolution des protocoles de sécurité

Protocole	Année	État
Ouvert	—	Aucun chiffrement : tout circule en clair sur les ondes
WEP	1997	**Cassé** : se contourne en quelques minutes, à proscrire
WPA	2003	Transition, obsolète
WPA2-Personnel (PSK)	2004	Encore très répandu, solide **si** le mot de passe est long et aléatoire
WPA2-Entreprise	2004	Un identifiant par utilisateur via un serveur d'authentification
WPA3	2018	Le standard actuel, résiste bien mieux aux attaques par dictionnaire

Les risques principaux

- **Mot de passe faible** : avec WPA2-Personnel, un attaquant qui capte la connexion d'un appareil peut tenter de deviner le mot de passe hors ligne, à grande vitesse. Un mot de passe court ou présent dans les dictionnaires tombe vite.
- **WPS** : la connexion par code PIN à 8 chiffres a des faiblesses de conception connues. Il faut le désactiver.
- **Interface d'administration** : un routeur avec le mot de passe administrateur par défaut est l'erreur la plus fréquente et la plus grave.
- **Firmware obsolète** : les routeurs ont des failles comme n'importe quel logiciel, et beaucoup ne sont jamais mis à jour.
- **Faux point d'accès** (*evil twin*) : un attaquant crée un réseau avec le même nom qu'un réseau légitime pour attirer les connexions.
- **Réseaux publics** : sur un Wi-Fi ouvert, les autres utilisateurs du même réseau peuvent tenter d'observer ou de manipuler le trafic non chiffré.

Sécuriser un réseau domestique ou un hotspot

Mesure	Pourquoi
Changer l'identifiant et le mot de passe administrateur	Les identifiants par défaut sont publiés en ligne
WPA3, ou WPA2-AES à défaut	Les protocoles plus anciens sont cassés
Mot de passe Wi-Fi d'au moins 15 caractères aléatoires	Résiste aux attaques par dictionnaire
Désactiver WPS	Faiblesse de conception connue
Mettre à jour le firmware	Corrige les failles connues
Désactiver l'administration depuis internet	Seul le réseau local doit pouvoir administrer

Mesure	Pourquoi
Réseau invité séparé	Les visiteurs n'accèdent pas à tes appareils
Isolation des clients	Les clients d'un réseau public ne se voient pas entre eux
Désactiver les services inutiles	Chaque service ouvert est une porte de plus

À RETENIR

Pour un **hotspot commercial** (une Wi-Fi zone avec portail de connexion, par exemple sur un routeur MikroTik), les mêmes principes s'appliquent avec quelques points en plus : changer l'utilisateur `admin` par défaut et lui donner un mot de passe fort, restreindre l'accès aux services d'administration (Winbox, SSH, interface web) au seul réseau d'administration, désactiver les services IP inutilisés, activer l'isolation entre clients, et maintenir RouterOS à jour. C'est un excellent sujet pour ton rapport du jour 29, puisque c'est un système que tu as le droit d'auditer.

Se protéger sur un réseau public

- Privilégier les sites et applications en **HTTPS** (c'est le cas de presque tous aujourd'hui).
- Utiliser les données mobiles plutôt qu'un Wi-Fi ouvert pour les opérations sensibles (banque, paiements).
- Ne jamais installer de « certificat » ou de profil demandé par un portail Wi-Fi.
- Désactiver la connexion automatique aux réseaux ouverts.
- Un **VPN** de confiance chiffre tout le trafic jusqu'au serveur VPN — mais il déplace la confiance vers le fournisseur du VPN, il ne la supprime pas.

EXERCICE

Connecte-toi à l'interface d'administration de **ton** routeur. Remplis ce tableau dans ton journal : chiffrement utilisé, WPS activé ou non, mot de passe admin changé ou non, version du firmware et date de la dernière mise à jour, administration distante activée ou non, nombre d'appareils connectés. Corrige ce qui doit l'être.

CHAPITRE 10 · JOUR 15

La méthodologie d'un test d'intrusion

Pourquoi une méthode ?

Ce qui distingue un professionnel d'un amateur, ce n'est pas l'outil : c'est la **méthode**. Une méthode garantit qu'on n'oublie rien, qu'on reste dans le périmètre, qu'on peut reproduire chaque résultat et qu'on peut l'expliquer au client. Plusieurs référentiels publics existent (PTES, OSSTMM, le guide de test de l'OWASP, le NIST SP 800-115). Ils se ressemblent tous dans les grandes lignes.

Les cinq phases

Phase	Question à laquelle elle répond	Exemples d'activités
0. Préparation	Qu'a-t-on le droit de faire ?	Contrat, périmètre, règles d'engagement
1. Reconnaissance	Que peut-on savoir de la cible ?	OSINT, DNS, recherche publique
2. Scan et énumération	Qu'est-ce qui est exposé, et en quelle version ?	Scan de ports, identification des services, cartographie d'un site
3. Exploitation	Les faiblesses sont-elles réellement exploitables ?	Démontrer une faille, de manière contrôlée
4. Post-exploitation	Quel est l'impact réel ?	Évaluer jusqu'où un attaquant pourrait aller
5. Rapport	Que doit corriger le client, et dans quel ordre ?	Rédaction, présentation, recommandations

À RETENIR

La phase la plus importante pour le client est le **rapport**. Une faille trouvée mais mal expliquée ne sera pas corrigée. Pour toi, la phase la plus importante est la **reconnaissance et l'énumération** : les professionnels répètent souvent que la qualité d'un test se joue à 80 % avant l'exploitation.

Les types de tests

Type	Ce que sait le testeur au départ	Simule…
Boîte noire	Rien, à part le nom ou l'adresse de la cible	Un attaquant extérieur
Boîte grise	Un compte utilisateur, une documentation partielle	Un client malveillant ou un compte compromis
Boîte blanche	Tout : code source, architecture, accès	Un audit le plus complet possible

Audit, pentest, Red Team, bug bounty

- Un **audit de configuration** vérifie qu'un système respecte de bonnes pratiques, sans forcément l'attaquer.
- Un **scan de vulnérabilités** est automatisé et produit une liste de failles potentielles, souvent avec des faux positifs.
- Un **test d'intrusion** va plus loin : un humain vérifie, combine et démontre l'impact réel des failles.
- Une **mission Red Team** teste l'organisation entière, y compris sa capacité à détecter et réagir.
- Le **bug bounty** est un test continu, ouvert à de nombreux chercheurs, payé à la faille trouvée.

La kill chain

Les défenseurs décrivent souvent une attaque comme une **chaîne** d'étapes (la *Cyber Kill Chain* de Lockheed Martin) : reconnaissance, préparation de l'arme, livraison, exploitation, installation, prise de contrôle à distance, actions sur l'objectif. Le message clé : **casser n'importe quel maillon arrête l'attaque**. C'est pourquoi la défense se fait à tous les niveaux.

Le référentiel **MITRE ATT&CK** va plus loin en recensant des centaines de techniques observées chez de vrais attaquants, classées par objectif (accès initial, persistance, élévation de privilèges, etc.). C'est un langage commun entre attaquants éthiques et défenseurs. Il est consultable gratuitement en ligne.

Prendre des notes : une compétence à part entière

Pendant un test, on note **tout** : chaque commande, chaque résultat intéressant, l'heure, les captures d'écran. Ces notes servent à rédiger le rapport, à reproduire un résultat, et à prouver ce qu'on a fait (et ce qu'on n'a pas fait) en cas de problème.

> **ASTUCE TÉLÉPHONE**
>
> Sur téléphone, une application de notes qui gère le Markdown (comme Obsidian ou Joplin) est idéale. Crée un modèle avec les rubriques : Cible · Périmètre · Reconnaissance · Ports et services · Pistes · Failles confirmées · Preuves · Recommandations.

> **EXERCICE**
>
> Pour chacune des cinq phases, écris dans ton journal une question qu'elle pose et un outil ou une source qu'on pourrait utiliser.

CHAPITRE 11 · JOUR 16

La reconnaissance et l'OSINT

Reconnaissance passive et active

	Passive	Active
Contact avec la cible	Aucun	Direct (scan, requêtes)
Détectable par la cible	Non	Oui
Sources	Publiques, tierces	Les systèmes de la cible
Exemples	Moteurs de recherche, DNS publics, registres	Nmap, navigation sur le site, énumération

OSINT (*Open Source Intelligence*) désigne le renseignement obtenu à partir de sources **publiques et légalement accessibles**. C'est le cœur de la reconnaissance passive.

Ce que l'on cherche sur une organisation

- **Les domaines et sous-domaines** : chaque sous-domaine (`dev.`, `test.`, `admin.`) est une porte potentielle, et les environnements de test oubliés sont souvent moins protégés.
- **Les adresses IP et l'hébergement** : quels serveurs, chez quel hébergeur.
- **Les technologies** : quel serveur web, quel CMS, quels frameworks.
- **Les personnes** : noms, postes, format des adresses e-mail — utiles pour évaluer le risque d'hameçonnage.
- **Les fuites** : documents publiés par erreur, code source public contenant des secrets, identifiants exposés dans des fuites

passées.

Les sources classiques

Source	Ce qu'on y trouve
WHOIS	Le registrar, les dates de création et d'expiration d'un domaine
Enregistrements DNS	Serveurs mail (MX), alias, vérifications (TXT)
Journaux de transparence des certificats (ex. crt.sh)	Les sous-domaines pour lesquels un certificat a été émis
Moteurs de recherche avancée	Fichiers et pages indexés précisément
Archives du web (Wayback Machine)	Anciennes versions d'un site, pages disparues
Dépôts de code publics	Code, et parfois des secrets publiés par erreur
Réseaux professionnels	Organigramme, technologies utilisées (via les offres d'emploi)
Moteurs de recherche d'objets connectés (Shodan, Censys)	Les services exposés sur internet, vus par leurs propres scans

Les opérateurs de recherche

Les moteurs de recherche acceptent des **opérateurs** qui précisent une recherche :

Opérateur	Effet
`site:exemple.com`	Résultats uniquement sur ce domaine
`filetype:pdf`	Seulement ce type de fichier
`intitle:"mot"`	Le mot dans le titre de la page
`inurl:admin`	Le mot dans l'adresse
`"expression exacte"`	Expression exacte

Opérateur	Effet
`-mot`	Exclure un mot

ATTENTION

Trouver une information publique est légal ; s'en servir pour accéder à un système, ou compiler un dossier sur une personne privée, ne l'est pas forcément. L'OSINT sur des **individus** pose de vraies questions de vie privée. En entraînement, travaille sur des organisations dans le cadre d'un exercice autorisé, ou **sur toi-même**.

Faire son propre audit d'exposition

C'est l'exercice le plus utile et le plus révélateur : appliquer l'OSINT à soi-même.

1. Cherche ton nom complet, ton pseudo, ton numéro de téléphone et ton adresse e-mail entre guillemets.
2. Vérifie sur un service de notification de fuites reconnu (comme *Have I Been Pwned*) si ton adresse e-mail apparaît dans des fuites connues.
3. Regarde ce que montrent tes profils publics à quelqu'un qui n'est pas ton contact.
4. Regarde les photos que tu publies : un arrière-plan peut révéler un lieu, une plaque, un badge.

EXERCICE

Fais ton audit d'exposition. Note tout ce qu'un inconnu pourrait apprendre de toi en 20 minutes. Change les mots de passe des comptes présents dans des fuites, et ajuste la visibilité de tes profils.

L'énumération : la reconnaissance active

Une fois la reconnaissance passive terminée, et **dans le périmètre autorisé**, vient l'énumération : scans de ports (chapitre 8), identification des versions, cartographie des pages et des répertoires d'un site, découverte des paramètres d'une application. On cherche à dresser la **liste la plus complète possible de la surface d'attaque**.

> **À RETENIR**
>
> Plus ta liste est complète, plus tu as de chances de trouver le point faible. Les débutants foncent sur la première piste ; les professionnels énumèrent tout avant de choisir où creuser.

CHAPITRE 12 · JOUR 17

Comment fonctionne une application web

Le modèle client-serveur

Une application web met en jeu au moins trois éléments :

1. **Le client** : ton navigateur, qui affiche les pages et exécute du code JavaScript.
2. **Le serveur web et l'application** : un programme (en PHP, Python, JavaScript, Java…) qui reçoit les requêtes et fabrique les réponses.
3. **La base de données** : où sont stockés utilisateurs, produits, messages.

> **À RETENIR**
>
> La règle d'or de la sécurité web : **tout ce qui vient du client peut être modifié par l'utilisateur**. Les champs de formulaire, les paramètres d'URL, les cookies, les en-têtes, les champs cachés, les vérifications faites en JavaScript dans le navigateur — tout. Une application sûre **revérifie tout côté serveur**.

Le front-end et le back-end

	Front-end	Back-end
S'exécute	Dans le navigateur	Sur le serveur
Langages	HTML, CSS, JavaScript	PHP, Python, Node.js, Java, Go…

	Front-end	Back-end
Visible par l'utilisateur	Oui, entièrement	Non
Peut être contourné	Oui, toujours	C'est là que doit se faire la sécurité

Les paramètres

Les applications reçoivent des données de plusieurs façons :

- dans l'**URL** (requête GET) : `https://boutique.exemple/produit?id=42&couleur=bleu` ;
- dans le **corps** de la requête (POST), typiquement lors de l'envoi d'un formulaire ;
- dans les **cookies** et les **en-têtes** ;
- au format **JSON**, pour les API utilisées par les applications mobiles et les sites modernes.

Chaque paramètre est un **point d'entrée** que tu devras tester lors d'un audit.

Les cookies et les sessions

HTTP est **sans état** : chaque requête est indépendante, le serveur ne « se souvient » pas de toi. Pour savoir que tu es connecté, l'application crée une **session** côté serveur et te donne un **identifiant de session**, stocké dans un **cookie**. Ton navigateur renvoie ce cookie à chaque requête.

Conséquence : **voler le cookie de session, c'est voler la session**. Celui qui le possède est considéré comme connecté à ta place, sans avoir besoin de ton mot de passe. C'est pourquoi les cookies de session doivent être protégés par des attributs :

Attribut	Protection
`Secure`	Le cookie n'est envoyé qu'en HTTPS

Attribut	Protection
`HttpOnly`	Le JavaScript de la page ne peut pas le lire (limite l'impact des XSS)
`SameSite`	Limite son envoi depuis d'autres sites (limite les attaques CSRF)

Les API

Les applications mobiles modernes (y compris celles sur ton téléphone) communiquent avec leurs serveurs via des **API** : des adresses qui reçoivent et renvoient du JSON plutôt que des pages HTML. Les API sont aujourd'hui l'une des surfaces d'attaque les plus importantes, car elles exposent directement la logique de l'application, et oublient souvent des contrôles que l'interface graphique, elle, faisait.

Le proxy d'interception

L'outil central de l'audit web est le **proxy d'interception** : un programme qui se place entre ton navigateur et le serveur, et te permet de **voir et modifier** chaque requête avant qu'elle parte. Le plus connu est **Burp Suite** (une édition gratuite existe), l'alternative libre est **OWASP ZAP**. Ils tournent sur ordinateur.

> **ASTUCE TÉLÉPHONE**
>
> Sur téléphone, tu peux déjà beaucoup observer : `view-source:` devant une URL dans Chrome affiche le code HTML ; `curl -v` dans Termux montre toute une requête et sa réponse. Et les laboratoires de PortSwigger peuvent se résoudre en partie dans le navigateur en modifiant les URL. Le proxy d'interception sera la première chose à apprendre le jour où tu auras un ordinateur.

Les en-têtes de sécurité

Un serveur peut demander au navigateur d'appliquer des protections supplémentaires grâce à des en-têtes :

En-tête	Rôle
`Strict-Transport-Security`	Forcer HTTPS pour toutes les visites futures
`Content-Security-Policy`	Limiter d'où peuvent venir les scripts (protection contre les XSS)
`X-Frame-Options`	Empêcher l'affichage dans un cadre d'un autre site (clickjacking)
`X-Content-Type-Options`	Empêcher le navigateur de deviner le type des fichiers

EXERCICE

Dans Termux, lance `curl -I` sur trois grands sites que tu utilises. Lesquels de ces en-têtes renvoient-ils ? Note les différences.

CHAPITRE 13 · JOUR 18

L'OWASP Top 10

Qu'est-ce que l'OWASP ?

L'**OWASP** (*Open Worldwide Application Security Project*) est une fondation à but non lucratif qui publie gratuitement des guides, des outils et des référentiels de sécurité applicative. Son document le plus célèbre est le **Top 10** : la liste des dix catégories de risques les plus critiques pour les applications web, établie à partir de données réelles. Il est mis à jour tous les quelques années ; vérifie la version en vigueur sur le site de l'OWASP.

> **À RETENIR**
>
> Le Top 10 n'est pas une liste de failles précises, mais de **catégories**. Chacune regroupe de nombreux types de vulnérabilités qui partagent la même cause profonde.

Les grandes catégories

Voici les catégories telles qu'elles figurent dans l'édition 2021, la référence la plus enseignée — l'édition suivante en reprend l'essentiel avec quelques réorganisations :

Code	Catégorie	Idée clé
A01	Contrôle d'accès défaillant	L'utilisateur peut faire ou voir des choses qui ne lui sont pas permises
A02	Défaillances cryptographiques	Données sensibles mal protégées : pas de chiffrement, algorithmes faibles

Code	Catégorie	Idée clé
A03	Injection	Des données de l'utilisateur sont interprétées comme du code (SQL, commandes, XSS)
A04	Conception non sécurisée	La faille est dans la logique même de l'application
A05	Mauvaise configuration	Options par défaut, messages d'erreur trop bavards, services inutiles
A06	Composants vulnérables ou obsolètes	Bibliothèques ou logiciels avec des failles connues
A07	Défaillances d'identification et d'authentification	Mots de passe faibles acceptés, sessions mal gérées, pas de MFA
A08	Défaut d'intégrité des logiciels et des données	Mises à jour ou données non vérifiées
A09	Journalisation et surveillance insuffisantes	L'attaque a lieu et personne ne s'en rend compte
A10	Falsification de requêtes côté serveur (SSRF)	On force le serveur à faire des requêtes à notre place

Quelques exemples pour fixer les idées

- **A01** : en changeant `/facture/1001` en `/facture/1002`, on voit la facture d'un autre client (c'est une IDOR, chapitre 15).
- **A02** : un site stocke les mots de passe en clair ; ou un formulaire de paiement fonctionne en HTTP.
- **A05** : une page d'erreur affiche la pile d'appels complète et la version du framework ; un panneau d'administration accessible avec `admin/admin`.
- **A06** : un site utilise une ancienne version de son CMS, affectée par une faille publiée depuis deux ans.
- **A09** : des milliers de tentatives de connexion ratées ne déclenchent aucune alerte.

Les protections qui reviennent toujours

Si tu ne retiens qu'une chose de ce chapitre, retiens ces principes, qui répondent à la majorité du Top 10 :

1. **Ne jamais faire confiance aux entrées** : valider, filtrer, et utiliser des mécanismes qui séparent données et code.
2. **Vérifier les droits côté serveur, à chaque requête.**
3. **Mettre à jour** systématiquement les composants.
4. **Configurer de façon minimale** : désactiver tout ce qui ne sert pas, changer tous les identifiants par défaut.
5. **Journaliser et surveiller.**

> **EXERCICE**
>
> Fais 10 fiches (une par catégorie) avec : le nom, un exemple que tu as inventé toi-même, et une protection. Relis-les pendant 5 minutes chaque jour jusqu'au jour 22.

CHAPITRE 14 · JOURS 19 ET 20

Les injections : SQL et XSS

Le principe de toute injection

Une injection se produit quand une application **mélange des données fournies par l'utilisateur avec du code**, sans les séparer correctement. L'utilisateur peut alors fabriquer une donnée qui sera **interprétée comme une instruction**. C'est la même cause profonde pour l'injection SQL, l'injection de commandes et le XSS ; seul le « langage » visé change.

Le langage SQL en deux minutes

SQL est le langage utilisé pour interroger les bases de données. Une base contient des **tables**, comme des tableurs :

```
SELECT nom, email FROM utilisateurs WHERE id = 42;
```

Cette requête signifie : « donne-moi le nom et l'e-mail de l'utilisateur dont l'identifiant est 42 ».

Comment naît une injection SQL

Imagine une application qui construit sa requête en **collant** directement ce que l'utilisateur a tapé dans le texte de la requête :

```
# CODE VULNÉRABLE — à ne jamais écrire
requete = "SELECT * FROM utilisateurs WHERE nom = '" + saisie
+ "'"
```

Tant que l'utilisateur tape un nom normal, tout va bien. Mais la saisie est insérée **telle quelle** au milieu du code SQL. Si elle contient des caractères qui ont un sens en SQL — une apostrophe, par exemple,

qui ferme la chaîne de texte — l'utilisateur peut **sortir** de la donnée et ajouter ses propres instructions à la requête. Selon les cas, il peut alors contourner une authentification, lire des tables entières, voire modifier ou supprimer des données.

Le premier signe d'une faille, pour un testeur, est souvent très simple : un caractère spécial comme l'apostrophe provoque une **erreur SQL** ou un comportement inattendu. Cela montre que la saisie a « cassé » la requête, donc qu'elle n'est pas traitée comme une simple donnée.

La vraie solution : les requêtes préparées

La protection n'est pas d'essayer de filtrer les « mauvais caractères » (on en oublie toujours). C'est de **séparer le code et les données** grâce aux **requêtes préparées** (ou *paramétrées*) :

```
# CODE SÛR — requête préparée (Python)
curseur.execute(
    "SELECT * FROM utilisateurs WHERE nom = %s",
    (saisie,)
)
```

La structure de la requête est envoyée d'abord, avec un emplacement réservé (`%s`). La saisie est envoyée **séparément**, et la base de données sait qu'il s'agit d'une valeur, jamais de code. Quoi que contienne la saisie, elle ne peut pas modifier la structure de la requête.

> **À RETENIR**
>
> Requêtes préparées partout + compte de base de données avec le **minimum de droits** + messages d'erreur génériques pour l'utilisateur = l'essentiel de la défense contre l'injection SQL.

L'injection de commandes

Même principe, mais avec le shell du système : une application qui construit une commande système en y collant une saisie utilisateur (par exemple pour lancer un `ping` vers une adresse fournie) peut permettre d'exécuter **d'autres commandes** sur le serveur. Les caractères comme `;`, `|` ou `&&` ont en effet un sens pour le shell — tu les as utilisés toi-même au chapitre 4. Protection : éviter d'appeler le shell, utiliser des fonctions qui passent les arguments séparément, et valider strictement les entrées (par exemple, n'accepter qu'une adresse IP bien formée).

Le XSS (Cross-Site Scripting)

Le **XSS** est une injection dans le **navigateur des autres utilisateurs**. L'application affiche une donnée fournie par un utilisateur sans la neutraliser, si bien que le navigateur l'interprète comme du HTML ou du JavaScript au lieu de l'afficher comme du texte. Le script s'exécute alors **dans la page du site**, avec ses droits : il peut lire des informations de la page, agir au nom de la victime, ou afficher un faux formulaire.

Type	Où est le code injecté	Exemple de lieu
Réfléchi	Dans la requête, renvoyé immédiatement dans la réponse	Page de résultats de recherche affichant le terme cherché
Stocké	Enregistré par le serveur, affiché à tous les visiteurs	Commentaires, profils, messages
Basé sur le DOM	Traité uniquement par le JavaScript de la page	Une page qui lit un fragment d'URL et l'insère dans la page

Le XSS **stocké** est généralement le plus grave : il touche automatiquement tous ceux qui consultent la page, sans qu'on ait besoin de leur faire cliquer sur un lien piégé.

Se protéger du XSS

1. **Échapper les sorties** : avant d'afficher une donnée, convertir les caractères spéciaux en leur équivalent texte (`<` devient `<`). Les moteurs de gabarits modernes le font automatiquement si on ne le désactive pas.
2. Utiliser les fonctions qui insèrent du **texte** et non du HTML (en JavaScript : `textContent` plutôt que `innerHTML`).
3. Mettre en place une **Content Security Policy** stricte.
4. Marquer les cookies de session `HttpOnly`.

ATTENTION

Tu t'entraîneras sur les injections **uniquement** dans les laboratoires de PortSwigger Web Security Academy, TryHackMe ou d'autres environnements prévus pour ça. Tester un formulaire d'un vrai site, même « juste avec une apostrophe », reste une tentative d'intrusion.

EXERCICE

Fais les leçons « SQL injection » et « Cross-site scripting » de PortSwigger Academy et au moins un lab *Apprentice* de chaque. Puis explique dans ton journal, avec tes mots, pourquoi les deux failles ont la même cause.

CHAPITRE 15 · JOUR 21

Authentification, sessions et contrôle d'accès

Authentification et autorisation : deux choses différentes

- L'**authentification** répond à « **qui es-tu ?** » (identifiant, mot de passe, code reçu par SMS, empreinte).
- L'**autorisation** (ou contrôle d'accès) répond à « **as-tu le droit de faire ça ?** ».

Une application peut parfaitement vérifier qui tu es, puis oublier de vérifier si tu as le droit de voir une page. C'est la catégorie numéro 1 de l'OWASP Top 10.

Les failles d'authentification courantes

- Mots de passe faibles acceptés, identifiants par défaut jamais changés.
- Aucune limite au nombre de tentatives : un attaquant peut essayer des milliers de mots de passe.
- Messages d'erreur différents selon que l'identifiant existe ou non (« utilisateur inconnu » / « mot de passe incorrect ») : cela permet de deviner quels comptes existent.
- Procédure de « mot de passe oublié » mal conçue (questions secrètes faciles, lien de réinitialisation prévisible ou réutilisable).
- Pas d'authentification multifacteur sur les comptes sensibles.

La gestion des sessions

Bonne pratique	Pourquoi
Identifiant de session long et aléatoire	Impossible à deviner
Nouvel identifiant après la connexion	Empêche la fixation de session
Expiration après inactivité	Limite l'impact d'un vol
Vraie déconnexion côté serveur	La session ne doit plus être valable après « Se déconnecter »
Cookies `Secure`, `HttpOnly`, `SameSite`	Voir chapitre 12

L'IDOR : la faille la plus simple et la plus fréquente

IDOR (*Insecure Direct Object Reference*) : l'application utilise un identifiant fourni par l'utilisateur pour accéder à un objet, **sans vérifier** que cet objet lui appartient.

Exemple fictif : tu consultes ton profil à l'adresse `https://app.exemple/api/profil?id=1057`. Si, en remplaçant `1057` par `1058`, l'application renvoie le profil de quelqu'un d'autre, c'est une IDOR. Aucun outil sophistiqué n'est nécessaire : c'est purement une question de **logique**. C'est pour cela que l'IDOR est une des failles les plus rapportées dans les programmes de bug bounty.

> **ATTENTION**
>
> C'est aussi exactement le genre de faille sur laquelle on peut tomber **par hasard** en utilisant un vrai service. Si cela t'arrive : tu t'arrêtes, tu ne consultes pas d'autres comptes, et tu suis la procédure de divulgation responsable du chapitre 2.

L'élévation de privilèges horizontale et verticale

- **Horizontale** : accéder aux données d'un **autre utilisateur du même niveau** (l'IDOR ci-dessus).
- **Verticale** : accéder à des fonctions d'un **niveau supérieur**, par exemple une page d'administration accessible à un simple utilisateur qui en connaît l'adresse.

La protection est toujours la même : **chaque requête** doit vérifier, côté serveur, que l'utilisateur connecté a le droit d'agir sur **cet objet précis**. Cacher un bouton dans l'interface ne protège rien.

L'authentification multifacteur (MFA)

La MFA combine plusieurs **facteurs** de natures différentes :

Facteur	Exemple
Ce que tu sais	Mot de passe, code PIN
Ce que tu possèdes	Téléphone avec application d'authentification, clé de sécurité physique
Ce que tu es	Empreinte digitale, visage

Les codes par **SMS** valent mieux que rien, mais restent vulnérables (détournement de numéro, hameçonnage). Les **applications d'authentification** et surtout les **clés de sécurité** et **passkeys** sont bien plus robustes, ces dernières résistant même à l'hameçonnage.

EXERCICE

Fais la leçon « Access control vulnerabilities » de PortSwigger Academy et un lab *Apprentice*. Puis active la double authentification par application (pas par SMS si possible) sur ton e-mail principal et tes réseaux sociaux.

CHAPITRE 16 · JOUR 23

Mots de passe et hachage

Comment un site devrait stocker ton mot de passe

Un site sérieux ne stocke **jamais** ton mot de passe. Il stocke son **empreinte** (*hash*), calculée par une **fonction de hachage**. Quand tu te connectes, il calcule l'empreinte de ce que tu as tapé et la compare à celle qu'il a enregistrée. Si elles sont identiques, c'est le bon mot de passe.

Les propriétés d'une fonction de hachage

1. **Sens unique** : à partir de l'empreinte, on ne peut pas « remonter » au mot de passe par calcul.
2. **Déterministe** : la même entrée donne toujours la même empreinte.
3. **Effet avalanche** : changer un seul caractère change complètement l'empreinte.
4. **Taille fixe** : quelle que soit la longueur de l'entrée, l'empreinte a toujours la même longueur.

```
echo -n "bonjour" | sha256sum
echo -n "Bonjour" | sha256sum
```

Fais l'essai dans Termux : une seule majuscule, et les deux empreintes n'ont rien en commun. L'option `-n` évite d'ajouter un retour à la ligne, qui changerait aussi l'empreinte.

Hachage, chiffrement, encodage : ne plus jamais confondre

	Encodage	Hachage	Chiffrement
But	Représenter des données dans un autre format	Vérifier sans révéler	Cacher, puis pouvoir relire
Réversible ?	Oui, par tout le monde	Non	Oui, avec la clé
Clé ?	Non	Non	Oui
Exemples	Base64, URL-encodage, hexadécimal	SHA-256, bcrypt, Argon2	AES, RSA, ChaCha20
Protège ?	**Non**	Oui (si bien choisi)	Oui

Alors, comment « casse-t-on » un mot de passe ?

Puisqu'on ne peut pas inverser une empreinte, l'attaquant qui a volé une base d'empreintes fait l'inverse : il **devine**. Il calcule l'empreinte de millions de mots de passe candidats et regarde si l'une correspond.

- **Attaque par dictionnaire** : on essaie des listes de mots de passe réels, issus de fuites passées. Les mots de passe humains se ressemblent énormément.
- **Attaque par règles** : on applique des transformations courantes aux mots du dictionnaire (majuscule au début, chiffres à la fin, `a` remplacé par `@`…).
- **Force brute** : on essaie toutes les combinaisons possibles. Efficace seulement sur les mots de passe courts.

Des outils comme **John the Ripper** et **Hashcat** sont utilisés par les auditeurs pour mesurer la robustesse des mots de passe d'une organisation, avec autorisation. Les plateformes d'entraînement proposent des empreintes de test pour apprendre à s'en servir.

Le sel et les algorithmes lents

Deux protections rendent ces attaques beaucoup plus difficiles :

1. Le **sel** : une valeur aléatoire différente pour chaque utilisateur, ajoutée au mot de passe avant le hachage. Deux personnes qui ont le même mot de passe n'ont plus la même empreinte, et les tables précalculées deviennent inutiles.
2. Les **algorithmes volontairement lents** : MD5 et SHA-1 (et même SHA-256 seul) sont conçus pour être **rapides**, ce qui est une catastrophe pour des mots de passe : on peut en tester des milliards par seconde. Les algorithmes dédiés aux mots de passe — **bcrypt**, **scrypt**, **Argon2** — sont conçus pour être lents et coûteux en mémoire, et réglables.

> **À RETENIR**
>
> Pour stocker des mots de passe : Argon2 ou bcrypt, avec un sel unique (géré automatiquement par ces algorithmes). Jamais MD5, jamais SHA-1, jamais en clair, jamais chiffré de façon réversible.

Ce qui fait un bon mot de passe

La **longueur** compte bien plus que la complexité. Une **phrase de passe** de quatre ou cinq mots choisis au hasard (« girafe-tambour-orage-citron-velo ») est à la fois plus longue, plus solide et plus facile à retenir que « P@ssw0rd1! ».

Mauvaise pratique	Bonne pratique
Le même mot de passe partout	Un mot de passe unique par site
Mots de passe courts et « complexes »	Longs : 15 caractères ou plus
Informations personnelles (date de naissance, prénom)	Aléatoire, généré par un outil
Tout retenir de tête	Un gestionnaire de mots de passe
Mot de passe seul	Avec une authentification multifacteur

La **réutilisation** est le vrai danger : quand un site se fait voler sa base, les attaquants essaient automatiquement les mêmes couples e-mail / mot de passe sur des centaines d'autres services. C'est le *credential stuffing*.

EXERCICE

Installe un gestionnaire de mots de passe reconnu (Bitwarden, KeePassDX sur Android, ou celui intégré à ton téléphone). Change en priorité le mot de passe de ton e-mail principal : c'est la clé qui permet de réinitialiser tous les autres.

CHAPITRE 17 · JOUR 24

Les bases de la cryptographie

Le vocabulaire

- **Texte clair** : le message lisible.
- **Texte chiffré** : le message rendu illisible.
- **Clé** : le secret qui permet de chiffrer et/ou de déchiffrer.
- **Algorithme** : la méthode. En cryptographie moderne, l'algorithme est **public** ; seule la clé est secrète (principe de Kerckhoffs). Un système dont la sécurité repose sur le secret de sa méthode est fragile.

Le chiffrement symétrique

Une **seule clé** sert à chiffrer et à déchiffrer. C'est rapide et efficace pour de gros volumes de données. L'algorithme de référence est **AES**.

Le problème : comment transmettre la clé à l'autre personne sans qu'elle soit interceptée ? Si tu dois déjà avoir un canal sûr pour envoyer la clé, autant envoyer directement le message par ce canal.

Le chiffrement asymétrique

Chaque personne possède une **paire de clés** mathématiquement liées :

- une **clé publique**, qu'on peut donner à tout le monde ;
- une **clé privée**, qu'on ne donne **jamais**.

Ce qui est chiffré avec la clé publique ne peut être déchiffré qu'avec la clé privée correspondante. Image : la clé publique est un cadenas ouvert que tu distribues à tout le monde ; n'importe qui peut fermer

une boîte avec, mais toi seul as la clé qui l'ouvre. Algorithmes courants : **RSA** et les courbes elliptiques (comme **Ed25519**).

La signature numérique

Dans l'autre sens : ce qui est signé avec la clé **privée** peut être vérifié par tous avec la clé **publique**. Cela prouve **qui** a signé et que le message **n'a pas été modifié**. C'est ce qui garantit l'authenticité des mises à jour logicielles, des certificats HTTPS ou des transactions.

Le meilleur des deux mondes

En pratique, on combine les deux. Dans une connexion HTTPS, la cryptographie asymétrique sert à **établir de façon sûre** une clé symétrique temporaire, puis tout l'échange est chiffré en symétrique, beaucoup plus rapide. Les messageries chiffrées de bout en bout fonctionnent sur le même principe.

Les certificats et les autorités

Comment être sûr qu'une clé publique appartient bien au site qu'on visite, et pas à un imposteur ? Grâce au **certificat** : un document qui associe un nom de domaine à une clé publique, **signé** par une **autorité de certification** à laquelle ton téléphone fait confiance. Si la signature n'est pas valide ou le nom ne correspond pas, le navigateur affiche un avertissement.

> **ATTENTION**
>
> Un avertissement de certificat sur un site que tu utilises d'habitude sans problème peut indiquer qu'on tente d'intercepter ta connexion. Ne clique jamais sur « continuer quand même » dans ce cas.

Les clés SSH en pratique

```
ssh-keygen -t ed25519
ls -la ~/.ssh
```

Tu obtiens deux fichiers : `id_ed25519` (la clé **privée**, protégée en `600`) et `id_ed25519.pub` (la clé **publique**). On place la clé publique sur un serveur pour s'y connecter sans mot de passe, de façon bien plus sûre. La clé privée ne quitte jamais ton appareil, et on peut la protéger elle-même par une phrase de passe.

Base64 : l'exemple type de fausse protection

```
echo "hacker" | base64
echo "aGFja2VyCg==" | base64 -d
```

Base64 sert à transporter des données binaires dans des systèmes qui n'acceptent que du texte. Ce n'est **pas** du chiffrement : n'importe qui peut décoder sans aucune clé. Pourtant, on trouve encore des applications qui « cachent » des mots de passe ou des identifiants en Base64. Le reconnaître (souvent des lettres, chiffres, `+`, `/`, et un ou deux `=` à la fin) est un réflexe de base en audit et dans les CTF.

> **AUTO-ÉVALUATION**
>
> Quelle est la différence entre hacher et chiffrer ? Pourquoi peut-on donner sa clé publique à tout le monde ? Pourquoi HTTPS utilise-t-il à la fois l'asymétrique et le symétrique ?

CHAPITRE 18 · JOUR 25

L'élévation de privilèges sous Linux

De quoi parle-t-on ?

Quand un attaquant obtient un premier accès à un système, c'est généralement avec les droits d'un **utilisateur limité** (par exemple le compte qui fait tourner le serveur web). L'**élévation de privilèges** consiste à passer de cet utilisateur limité à un utilisateur plus puissant, idéalement **root**. C'est l'étape qui transforme un incident limité en compromission complète.

Pour le défenseur, comprendre l'élévation de privilèges, c'est savoir **quelles erreurs de configuration ne jamais laisser** sur un serveur.

Les grandes familles de causes

Cause	Exemple d'erreur
Droits sudo trop larges	Un utilisateur peut lancer avec sudo un programme qui permet, lui, d'ouvrir un shell
Fichiers SUID mal choisis	Un programme qui s'exécute avec les droits de son propriétaire (root) alors qu'il ne devrait pas
Permissions trop ouvertes	Un script exécuté régulièrement par root, mais modifiable par tout le monde
Tâches planifiées (cron)	Une tâche root qui appelle un fichier situé dans un dossier accessible en écriture
Secrets en clair	Mots de passe dans des fichiers de configuration, des scripts ou l'historique des commandes

Cause	Exemple d'erreur
Logiciels non mis à jour	Un noyau ou un service affecté par une faille d'élévation connue
Réutilisation de mots de passe	Le mot de passe de la base de données est aussi celui de root

Le bit SUID

Normalement, un programme s'exécute avec les droits de **celui qui le lance**. Un fichier marqué **SUID** (un `s` à la place du `x` du propriétaire dans `ls -l`, par exemple `-rwsr-xr-x`) s'exécute avec les droits de son **propriétaire**. C'est utile pour certains programmes système, comme `passwd`, qui doit modifier un fichier réservé à root. Mais si un programme SUID root permet d'une manière ou d'une autre d'exécuter des commandes ou de lire n'importe quel fichier, tout utilisateur peut en profiter.

Comment un auditeur vérifie un système

Sur une machine qu'il a le droit d'auditer, un professionnel commence par des vérifications simples, qui sont aussi celles que tout administrateur devrait faire régulièrement :

```
id                          # qui suis-je, quels groupes ?
sudo -l                     # que puis-je lancer avec sudo ?
find / -perm -4000 -type f 2>/dev/null   # fichiers SUID
ls -la /etc/cron*           # tâches planifiées
cat ~/.bash_history         # commandes passées
uname -r                    # version du noyau
```

Le `2>/dev/null` masque les messages d'erreur « permission refusée », très nombreux quand on parcourt tout le système. Des scripts d'énumération automatique existent pour rassembler ces informations ; ils sont très utilisés dans les CTF.

Se défendre

- Principe du **moindre privilège** : chaque compte et chaque service n'a que les droits strictement nécessaires.
- Règles `sudo` limitées à des commandes précises, jamais à des programmes polyvalents.
- Revue régulière des fichiers SUID et des permissions des scripts exécutés par root.
- Aucun secret en clair dans les fichiers ; utiliser des coffres à secrets.
- Mises à jour régulières, en particulier du noyau.

À RETENIR

L'élévation de privilèges est rarement une prouesse technique. C'est presque toujours **une négligence de configuration** qu'on trouve parce qu'on a énuméré méthodiquement. Les mêmes réflexes que pour un scan réseau : tout lister, puis réfléchir.

EXERCICE

Termine Bandit jusqu'au niveau 15 au moins (plusieurs niveaux reposent sur ces notions), ou fais une salle TryHackMe d'introduction à l'élévation de privilèges sous Linux. Note pour chaque cas l'**erreur de configuration** et la **correction** qu'un administrateur aurait dû appliquer.

CHAPITRE 19 · JOUR 26

Penser en défenseur : la Blue Team

Pourquoi un attaquant éthique doit penser en défenseur

Le but final d'un test d'intrusion, c'est la **défense**. Un pentesteur qui ne comprend pas comment on protège et comment on détecte écrira des recommandations vagues et inutiles. À l'inverse, les meilleurs défenseurs sont ceux qui savent comment les attaquants raisonnent.

La défense en profondeur

Aucune protection n'est parfaite. On empile donc **plusieurs couches**, de sorte que si l'une cède, les suivantes arrêtent ou ralentissent l'attaque — comme un château fort : douves, remparts, portes, gardes, donjon.

Couche	Exemples de mesures
Humaine	Sensibilisation, procédures, double validation des virements
Physique	Accès aux locaux, salles serveurs fermées
Périmètre	Pare-feu, filtrage des e-mails
Réseau	Segmentation, réseaux séparés par usage
Machines	Mises à jour, antivirus/EDR, configuration durcie
Applications	Développement sécurisé, tests, revue de code
Données	Chiffrement, sauvegardes, contrôle d'accès

Les journaux (logs)

Les **journaux** enregistrent ce qui se passe : connexions, erreurs, requêtes web, commandes d'administration. Sans journaux, une attaque passe inaperçue, et on ne peut ni la comprendre ni la stopper. Sous Linux, ils se trouvent en général dans `/var/log` : par exemple les journaux d'authentification et ceux du serveur web.

Dans un journal web, des milliers de requêtes en quelques minutes depuis la même adresse, des pages qui n'existent pas demandées en rafale, ou des caractères inhabituels dans les paramètres sont autant de signes d'un scan ou d'une tentative d'attaque.

Le SOC et le SIEM

En entreprise, les journaux de toutes les machines sont centralisés dans un **SIEM**, un outil qui les rassemble, les corrèle et déclenche des **alertes**. Les analystes du **SOC** surveillent ces alertes, les qualifient (vraie attaque ou faux positif ?) et réagissent. Analyste SOC de niveau 1 est l'une des portes d'entrée les plus courantes dans la cybersécurité.

La réponse à incident

Quand une attaque est détectée, on suit un processus :

1. **Préparation** : avoir un plan avant que ça arrive.
2. **Détection et analyse** : que se passe-t-il, depuis quand, quelle ampleur ?
3. **Confinement** : isoler les machines touchées pour limiter la propagation.
4. **Éradication** : supprimer la cause (compte compromis, logiciel malveillant, faille).
5. **Rétablissement** : remettre en service, à partir de sauvegardes saines si besoin.

6. **Retour d'expérience** : comprendre et améliorer pour la prochaine fois.

Les règles d'hygiène de base

Une grande partie des attaques réussies exploite des négligences élémentaires. Les mesures suivantes bloquent l'essentiel des attaques opportunistes :

- mettre à jour systèmes et applications rapidement ;
- authentification multifacteur partout où c'est possible ;
- sauvegardes régulières, testées, et dont une copie est **hors ligne** (contre les rançongiciels) ;
- moindre privilège pour tous les comptes ;
- inventaire de ce qui est exposé sur internet ;
- sensibilisation des utilisateurs.

EXERCICE

Reprends les attaques vues ce mois-ci (scan, injection SQL, XSS, IDOR, mots de passe, élévation de privilèges). Pour chacune, écris : quelle trace elle laisserait dans les journaux, et quelle mesure l'aurait empêchée.

CHAPITRE 20 · JOUR 27

L'ingénierie sociale

Pirater l'humain

L'**ingénierie sociale** consiste à manipuler une personne pour qu'elle fasse quelque chose qu'elle ne devrait pas : donner un mot de passe, cliquer sur un lien, ouvrir une pièce jointe, faire un virement. C'est souvent bien plus facile que d'attaquer une machine, car on exploite des réflexes humains normaux : la confiance, la serviabilité, la peur, l'urgence, la curiosité.

Les leviers psychologiques

Levier	Exemple
Urgence	« Votre compte sera suspendu dans 24 heures »
Autorité	Un faux message du directeur, de la banque, de la police
Peur	« Une activité suspecte a été détectée sur votre compte »
Appât du gain	« Vous avez gagné », « remboursement en attente »
Curiosité	« Voici les photos de la soirée », un colis inattendu
Serviabilité	Un faux technicien qui a « besoin d'un petit coup de main »

Les formes courantes

- **Hameçonnage (phishing)** : un e-mail imitant un service connu, avec un lien vers une fausse page de connexion.
- **Smishing** : la même chose par SMS (faux livreur, fausse banque, faux opérateur de paiement mobile).
- **Vishing** : par appel téléphonique, souvent avec un faux « service client » ou « service fraude ».

- **Spear phishing** : un hameçonnage **ciblé**, préparé grâce à l'OSINT, beaucoup plus crédible.
- **Fraude au président** : un faux dirigeant demande un virement urgent et confidentiel.
- **Prétexte** : un scénario inventé pour justifier une demande (« je suis le nouveau stagiaire de l'informatique »).

Reconnaître un message piège

1. L'**expéditeur** ne correspond pas exactement au service (adresse ou numéro inhabituel, domaine légèrement modifié).
2. Le **lien**, quand on appuie longuement dessus sans l'ouvrir, mène vers un domaine différent ou bizarre.
3. Une **urgence** ou une **menace** artificielle.
4. Une **demande inhabituelle** : code reçu par SMS, mot de passe, paiement, installation d'une application.
5. Des **fautes** ou une mise en forme étrange (même si les messages générés aujourd'hui sont de plus en plus soignés).
6. Un **ton** qui ne correspond pas à la personne ou au service habituel.

> **À RETENIR**
>
> La règle d'or : en cas de doute, **ne réponds pas par le canal reçu**. Contacte le service ou la personne par un moyen que tu connais déjà (application officielle, numéro habituel). Aucun service légitime ne te demandera jamais un code reçu par SMS.

L'ingénierie sociale dans les tests autorisés

Les entreprises font tester la vigilance de leurs employés par des **campagnes d'hameçonnage simulées**, menées par des professionnels avec un cadre très strict : autorisation de la direction, périmètre précis, pas de collecte réelle de mots de passe, objectif de **formation** et non de sanction. Ce type de test n'est jamais quelque

chose qu'on fait de sa propre initiative.

EXERCICE

Retrouve trois messages suspects que tu as reçus (e-mails, SMS, messages WhatsApp). Sans cliquer, liste pour chacun les indices qui trahissent la tentative. Puis explique ces indices à un proche : c'est souvent eux qui sont visés.

CHAPITRE 21 · JOURS 28 ET 29

CTF et rédaction de rapport

Les CTF

Un **CTF** (*Capture The Flag*) est une compétition ou un exercice où il faut résoudre des défis de sécurité pour trouver des « drapeaux » — des chaînes de texte cachées, souvent au format `flag{...}` ou `picoCTF{...}`. Les catégories classiques :

Catégorie	Contenu
Web	Failles d'applications web
Crypto	Chiffrements faibles, encodages, mauvaises implémentations
Forensics	Analyse de fichiers, d'images, de captures réseau
Reverse	Comprendre le fonctionnement d'un programme sans son code source
Pwn	Exploitation de programmes vulnérables (avancé)
OSINT	Retrouver une information à partir de sources publiques
Misc / General	Linux, scripts, logique

ASTUCE TÉLÉPHONE

picoCTF propose un terminal web dans le navigateur : tu peux résoudre beaucoup de défis débutants entièrement depuis ton téléphone. Commence par la catégorie *General Skills*, puis *Cryptography* et *Web Exploitation* de niveau facile.

La méthode pour un défi

1. **Lis entièrement** l'énoncé et les indices : tout est souvent dedans.
2. **Énumère** : de quoi dispose-t-on ? fichier, adresse, port, texte encodé ?
3. **Identifie** : quel type de fichier (`file`), quel encodage, quelle technologie ?
4. **Formule une hypothèse** et teste-la.
5. **Note** chaque essai, y compris les échecs.
6. Après 30 minutes bloqué : un **indice**, pas la solution.
7. Après la résolution : lis les solutions (*write-ups*) des autres, tu y apprendras d'autres approches.

Pourquoi le rapport est si important

Le **rapport** est le seul produit livré au client. Il doit être utile à deux publics très différents : la **direction**, qui veut savoir quel est le niveau de risque et quoi décider, et l'**équipe technique**, qui doit comprendre et corriger chaque faille.

La structure d'un rapport

Partie	Contenu
Page de garde	Titre, client, dates, auteur, niveau de confidentialité
Synthèse pour la direction	Une page, sans jargon : niveau de risque global, principaux constats, priorités
Périmètre et méthodologie	Ce qui a été testé, comment, ce qui était exclu, les limites
Tableau des vulnérabilités	Liste résumée avec le niveau de gravité
Détail de chaque vulnérabilité	Description, impact, preuves, étapes de reproduction, recommandation
Plan d'action	Ordre de correction suggéré

Partie	Contenu
Annexes	Résultats bruts, liste des outils

Évaluer la gravité

On classe généralement les failles en **critique**, **élevée**, **moyenne**, **faible** et **informative**. Le système **CVSS** fournit une note de 0 à 10 calculée à partir de critères objectifs : facilité d'exploitation, besoin d'être authentifié, besoin d'une action de la victime, impact sur la confidentialité, l'intégrité et la disponibilité (la triade CIA du chapitre 1). Une calculatrice CVSS gratuite est disponible en ligne sur le site du FIRST.

Rédiger une vulnérabilité : un modèle

À RETENIR

Titre : Mot de passe administrateur par défaut sur le routeur

Gravité : Critique

Description : L'interface d'administration du routeur est accessible depuis le réseau Wi-Fi avec l'identifiant et le mot de passe d'usine, publiés dans la documentation du fabricant.

Impact : Toute personne connectée au Wi-Fi peut prendre le contrôle total du routeur : modifier la configuration, intercepter ou rediriger le trafic, bloquer le réseau.

Preuve : capture de l'écran d'administration obtenu (sans modification effectuée).

Recommandation : Changer immédiatement l'identifiant et le mot de passe d'administration ; restreindre l'accès à l'interface d'administration ; désactiver l'administration depuis internet.

Les règles d'écriture

- Être **factuel** : ce qui a été observé, pas ce qu'on suppose.
- Être **reproductible** : quelqu'un d'autre doit pouvoir constater la faille en suivant tes étapes.
- Être **actionnable** : chaque recommandation doit dire concrètement quoi faire.
- Être **respectueux** : on décrit des failles de systèmes, on ne juge pas les personnes.
- Être **confidentiel** : un rapport de pentest est une carte des points faibles d'une organisation. Il se protège.

EXERCICE

Rédige le rapport de l'audit de ton propre réseau (jours 12 et 13) avec cette structure : une page de synthèse, puis une fiche par constat. Fais-le relire par quelqu'un de non technique : comprend-il ce qu'il doit faire ?

CHAPITRE 22 · JOUR 30

Après les 30 jours : construire ta carrière

Où tu en es

Si tu as suivi le programme, tu sais maintenant te servir d'un terminal Linux, tu comprends comment fonctionnent les réseaux et le web, tu connais la méthodologie d'un test d'intrusion et les grandes familles de failles, tu sais penser en défenseur et rédiger un rapport. C'est le socle sur lequel tout le reste se construit. Tu n'es pas encore un professionnel — mais tu n'es plus un débutant perdu.

Un parcours possible sur six mois

Mois	Objectif	Ressources
1 – 2	Consolider Linux et réseaux	Bandit complet, parcours débutants TryHackMe
2 – 3	Approfondir le web	Tous les labs *Apprentice* de PortSwigger, puis quelques *Practitioner*
3 – 4	Apprendre un langage de script	Python (automatiser, comprendre les outils), Bash
4 – 5	Machines complètes	Machines « faciles » de TryHackMe et Hack The Box, Root Me
5 – 6	Se spécialiser et se montrer	Write-ups publiés, CTF en équipe, première certification

ASTUCE TÉLÉPHONE

Apprendre **Python** est possible entièrement depuis Termux (`pkg install python`). C'est le langage le plus utile en sécurité : il te permettra d'écrire tes propres petits outils, de comprendre ceux des autres et d'automatiser les tâches répétitives.

L'obstacle de l'ordinateur

Tu peux aller remarquablement loin avec un téléphone : Linux, réseaux, théorie, labs web, CTF, Python. Mais certaines étapes demandent un ordinateur : proxy d'interception comme Burp Suite, machines virtuelles, connexion VPN aux machines de Hack The Box, examens pratiques des certifications. Quelques pistes réalistes :

- un ordinateur **d'occasion** même ancien suffit pour Linux (une distribution légère fonctionne sur des machines de 8 à 10 ans) ;
- les **cybercafés**, **bibliothèques**, **universités** et **espaces numériques** donnent parfois accès à des postes ;
- la **machine d'attaque dans le navigateur** de TryHackMe permet de travailler sur un vrai bureau Linux à distance ;
- un **clavier Bluetooth** avec Termux améliore énormément le confort en attendant.

Les certifications

Les certifications ne remplacent pas les compétences, mais elles ouvrent des portes, surtout pour un premier emploi. Les noms, contenus et prix évoluent : vérifie toujours sur le site officiel de l'organisme.

Niveau	Exemples	Profil
Fondations	CompTIA Security+, ISC2 Certified in Cybersecurity (CC)	Bases théoriques larges, reconnues par les recruteurs

Niveau	Exemples	Profil
Pratique débutant	eJPT (INE), certifications de TryHackMe	Premier examen pratique de pentest
Pratique intermédiaire	PNPT (TCM Security), CPTS (Hack The Box)	Examens pratiques réalistes, rapport exigé
Référence	OSCP (OffSec)	Très reconnue, examen pratique exigeant
Web	BSCP (PortSwigger)	Spécialisée sécurité web

Se construire un portfolio

Sans diplôme ni expérience, c'est ton **travail visible** qui convaincra :

- publie des **write-ups** de défis ou de machines **retirées** (les plateformes interdisent de publier les solutions des machines actives) ;
- tiens un petit blog ou un dépôt GitHub de tes notes et de tes scripts ;
- affiche ton profil et ta progression TryHackMe ou Root Me ;
- participe à des CTF en équipe et mentionne tes classements.

Les communautés

Apprendre seul est possible ; apprendre avec d'autres est plus rapide et plus motivant. Les plateformes d'entraînement ont presque toutes un serveur Discord actif. Les communautés francophones existent aussi : **Root Me** est une plateforme française très riche, et de nombreux pays d'Afrique francophone ont des associations, des clubs universitaires et des CTF nationaux en plein développement. Cherche ceux de ton pays.

Le mot de la fin

Tu as commencé ce parcours en partant de zéro. Trente jours plus tard, tu sais des choses que la plupart des utilisateurs d'internet ignorent toute leur vie. L'assurance que tu cherchais ne viendra pas d'un jour où tu te sentiras enfin « prêt » : elle vient, et viendra, de **l'accumulation de petites victoires**, notées jour après jour dans ton journal.

Continue à pratiquer un peu chaque jour. Reste curieux. Reste du bon côté de la loi. Et souviens-toi que le but de tout ce que tu apprends est de **rendre le numérique plus sûr pour tout le monde**.

OBJECTIF

Écris ton engagement pour les 90 prochains jours : plateforme choisie, nombre d'heures par semaine, objectif mesurable. Fixe une date dans ton agenda pour faire le point.

ANNEXE A

Aide-mémoire des commandes

Navigation et fichiers

Commande	Rôle
`pwd`	Afficher le dossier courant
`ls -la`	Lister tout, en détail
`cd dossier` / `cd ..` / `cd ~`	Se déplacer
`mkdir -p a/b/c`	Créer des dossiers imbriqués
`touch f`	Créer un fichier vide
`cp src dest` / `mv src dest`	Copier / déplacer ou renommer
`rm f` / `rm -r dossier`	Supprimer (définitivement)
`cat f` / `less f`	Afficher / lire page par page
`head -n 10 f` / `tail -n 10 f`	Début / fin d'un fichier
`nano f`	Éditer un fichier
`file f`	Type d'un fichier
`strings f`	Texte lisible d'un fichier binaire

Recherche et traitement de texte

Commande	Rôle
`grep -i mot f`	Chercher un mot (sans casse)
`grep -r mot dossier`	Chercher récursivement
`find . -name "*.txt"`	Chercher des fichiers par nom
`wc -l f`	Compter les lignes

Commande	Rôle
`sort f` / `uniq`	Trier / dédoublonner
`cut -d: -f1 f`	Extraire une colonne
`cmd1 | cmd2`	Enchaîner deux commandes
`cmd > f` / `cmd >> f`	Écrire / ajouter dans un fichier

Utilisateurs et permissions

Commande	Rôle
`whoami` / `id`	Identité
`chmod 755 f` / `chmod +x f`	Modifier les permissions
`chown user:groupe f`	Changer le propriétaire
`sudo -l`	Droits sudo de l'utilisateur

Réseau

Commande	Rôle
`ip addr` / `ifconfig`	Adresses de l'appareil
`curl ifconfig.me`	Adresse IP publique
`ping -c 4 hote`	Tester la joignabilité
`nslookup nom` / `dig nom`	Requête DNS
`curl -I url` / `curl -v url`	En-têtes HTTP / dialogue complet
`ssh user@hote -p port`	Connexion à distance
`nmap -sT -sV cible`	Scan de ports et versions (cible autorisée !)
`nmap -sn 192.168.1.0/24`	Découverte des appareils du réseau local

Hachage et encodage

Commande	Rôle
`echo -n "txt" | sha256sum`	Empreinte SHA-256
`echo "txt" | base64`	Encoder en Base64
`echo "dHh0Cg==" | base64 -d`	Décoder
`ssh-keygen -t ed25519`	Générer une paire de clés SSH

Termux

Commande	Rôle
`pkg update && pkg upgrade`	Mettre à jour
`pkg install nom` / `pkg search nom`	Installer / chercher
`termux-setup-storage`	Accès au stockage du téléphone

ANNEXE B

Glossaire

Terme	Définition
API	Interface permettant à des programmes de communiquer entre eux, souvent en JSON
Authentification	Prouver son identité
Autorisation	Avoir le droit de faire une action
Bug bounty	Programme qui récompense la découverte de failles sur un périmètre autorisé
CIA (DIC)	Confidentialité, Intégrité, Disponibilité
CIDR	Notation `/n` indiquant la taille d'un réseau
Cookie	Petite donnée stockée par le navigateur et renvoyée au site
CTF	Compétition de défis de sécurité (Capture The Flag)
CVE	Identifiant public d'une vulnérabilité connue (ex. CVE-2024-XXXX)
CVSS	Système de notation de la gravité d'une faille, de 0 à 10
Défense en profondeur	Empilement de protections successives
DNS	Système qui traduit les noms de domaine en adresses IP
Encodage	Changement de format réversible, sans secret
Énumération	Inventaire méthodique de la surface d'attaque
Exploit	Technique ou programme qui tire parti d'une vulnérabilité
Hachage	Fonction à sens unique produisant une empreinte
HTTPS	HTTP chiffré par TLS

Terme	Définition
IDOR	Accès à un objet d'autrui en changeant un identifiant
Injection	Donnée utilisateur interprétée comme du code
IP	Adresse identifiant une machine sur un réseau
MAC	Adresse physique d'une carte réseau
MFA	Authentification multifacteur
NAT	Traduction des adresses privées en adresse publique
OSINT	Renseignement à partir de sources publiques
OWASP	Fondation qui publie des référentiels de sécurité web
Pare-feu	Équipement ou logiciel qui filtre le trafic
Pentest	Test d'intrusion autorisé
Phishing	Hameçonnage : message frauduleux imitant un service légitime
Port	Numéro identifiant un service sur une machine
Privilège	Niveau de droits d'un utilisateur ou d'un processus
Root	Administrateur tout-puissant sous Linux
Scope	Périmètre autorisé d'un test
Sel	Valeur aléatoire ajoutée avant le hachage d'un mot de passe
Session	Mécanisme qui maintient la connexion d'un utilisateur
Shell	Programme qui interprète les commandes
SIEM	Outil qui centralise et analyse les journaux
SOC	Centre opérationnel de sécurité
SQL	Langage d'interrogation des bases de données
SSH	Protocole d'accès à distance chiffré
SUID	Bit qui fait s'exécuter un programme avec les droits de son propriétaire

Terme	Définition
TCP / UDP	Protocoles de transport, fiable / rapide
TLS	Protocole de chiffrement des communications
Vulnérabilité	Faiblesse pouvant être exploitée
XSS	Injection de script dans les pages vues par d'autres utilisateurs

ANNEXE C

Corrigés des questions de validation

Phase 1

Jour 1 — Pirate et pentesteur peuvent avoir les mêmes compétences ; le pentesteur agit avec une autorisation écrite, dans un périmètre défini, pour aider à corriger.

Jour 2 — Exemples : tester le Wi-Fi d'un voisin ; scanner le site de son école ou de son employeur sans accord de la direction ; essayer des mots de passe sur le compte d'un ami, même avec sa permission orale si le service ne l'autorise pas.

Jour 6 — `chmod 750` : le propriétaire lit, écrit et exécute (7) ; le groupe lit et exécute (5) ; les autres n'ont aucun droit (0).

Phase 2

Jour 8 — L'adresse IP est en couche 3 (réseau), l'adresse MAC en couche 2 (liaison), HTTP en couche 7 (application).

Jour 9 — Un `/24` compte 256 adresses, dont 254 utilisables.

Jour 10 — 22 : SSH ; 80 : HTTP ; 443 : HTTPS ; 3306 : MySQL ; 3389 : RDP.

Jour 11 — 200 : succès ; 301 : redirection permanente ; 403 : accès interdit ; 404 : introuvable ; 500 : erreur interne du serveur.

Jour 12 — *open* : un service accepte la connexion ; *closed* : la machine répond qu'aucun service n'écoute ; *filtered* : pas de réponse exploitable, probablement à cause d'un pare-feu.

Phase 3

Jour 15 — Pour le client, le rapport : c'est lui qui permet de corriger.

Jour 16 — La reconnaissance passive n'envoie rien vers les systèmes de la cible : elle n'utilise que des sources publiques et tierces.

Jour 17 — Le cookie de session est stocké par le navigateur. Le voler permet de se faire passer pour l'utilisateur connecté sans connaître son mot de passe.

Jour 19 — La requête préparée envoie la structure de la requête et les données séparément : la base sait que la saisie est une valeur et ne l'interprète jamais comme du code.

Jour 20 — Le XSS réfléchi est renvoyé dans la réponse à une requête piégée (il faut faire cliquer la victime) ; le stocké est enregistré par le serveur et touche tous les visiteurs. Le stocké est généralement plus grave.

Jour 21 — Exemple : `/api/commandes?id=5512` montre la commande d'un autre client si on remplace l'identifiant.

Phase 4

Jour 23 — Par l'effet avalanche : les fonctions de hachage sont conçues pour que la moindre modification change complètement le résultat.

Jour 24 — Base64 se décode sans aucune clé : c'est un format, pas une protection.

Jour 25 — Droits sudo trop larges, programmes SUID inadaptés, scripts exécutés par root mais modifiables par tous (et aussi : mots de passe en clair, logiciels non mis à jour).

Jour 26 — Superposer plusieurs protections indépendantes pour qu'aucune défaillance unique ne suffise à compromettre le système (comme les remparts successifs d'un château).

Jour 27 — Expéditeur suspect, lien vers un domaine inattendu, urgence ou menace, demande d'information sensible ou de paiement, ton ou forme inhabituels.